Introduction to Artificial Intelligence

Semester I, 2025-26

Markov Decision Processes

Rohan Paul

Outline

- Last Class
  - Modeling Uncertainty
- This Class
  - Markov Decision Processes
- Reference Material
  - Please follow the notes as the primary reference on this topic. Supplementary reading on topics covered in class from AIMA Ch 17 sections 17.1 – 17.3.

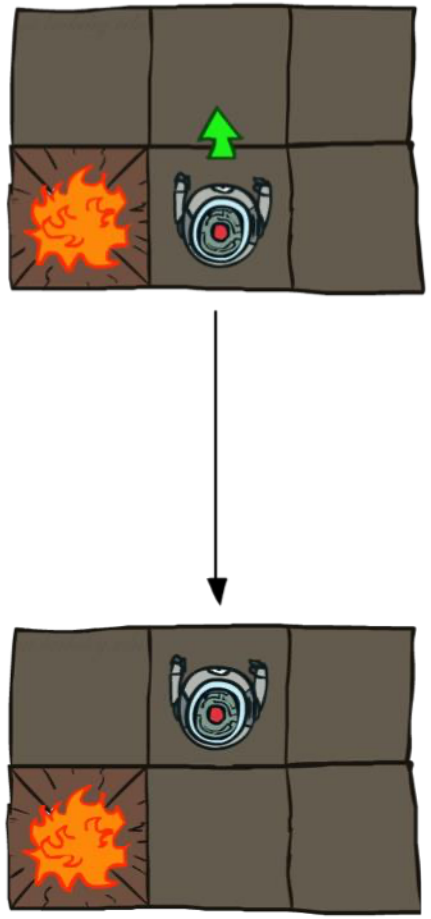
Acknowledgement

These slides are intended for teaching purposes only. Some material has been used/adapted from web sources and from slides by Doina Precup, Dorsa Sadigh, Percy Liang, Mausam, Dan Klein, Anca Dragan, Nicholas Roy, Emilio Frazzoli and others.

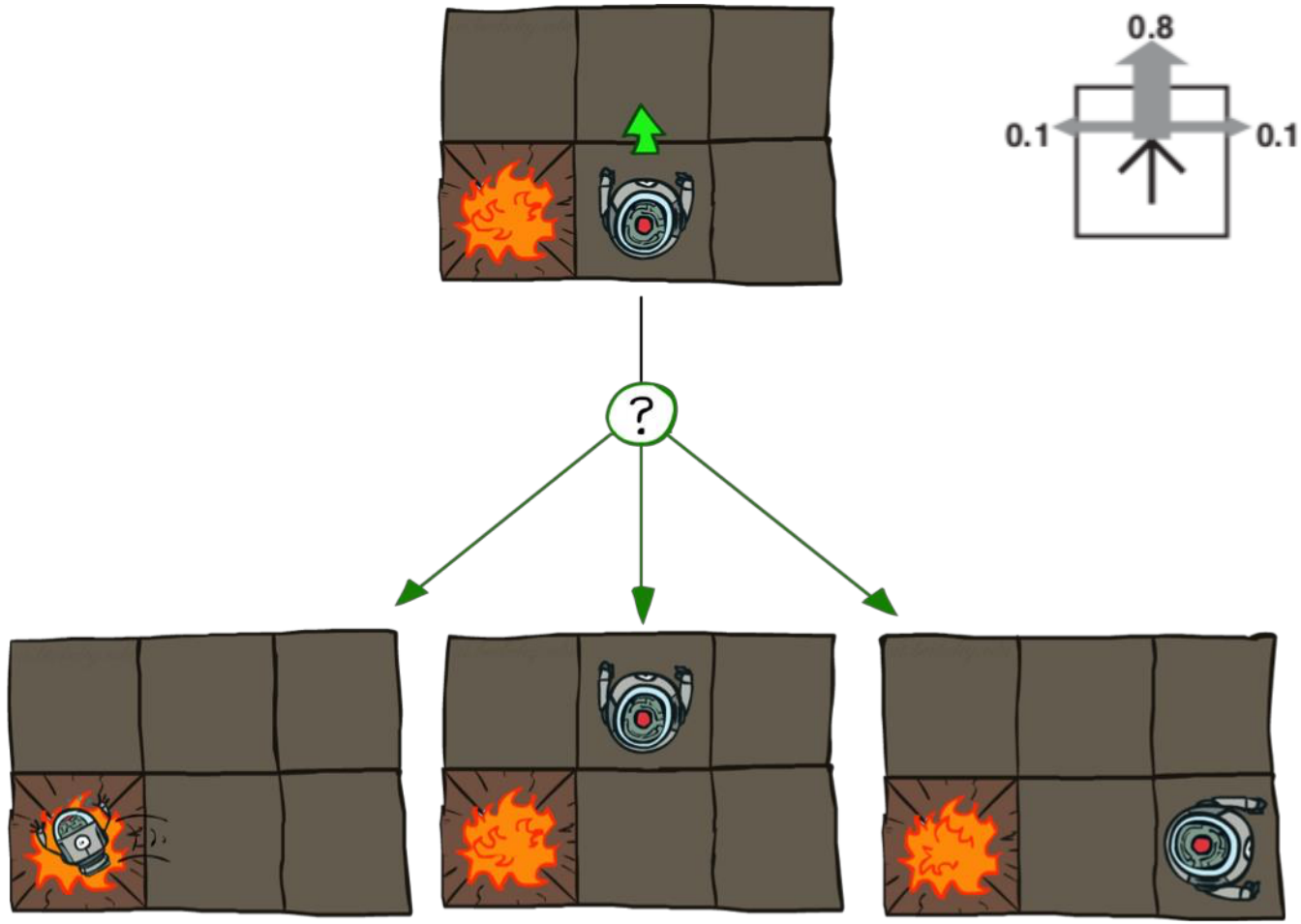
How to plan when actions are uncertain?

Consequences of Uncertainty in Actions

Deterministic Action Outcomes



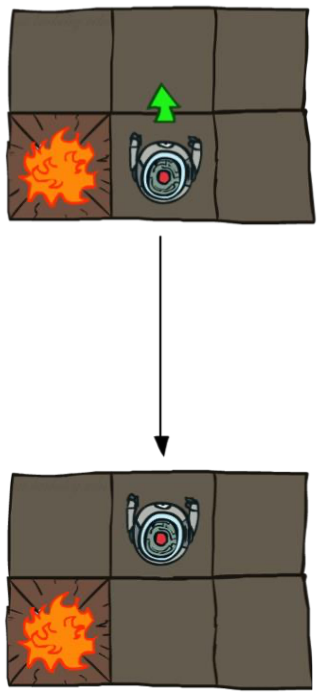
Stochastic Action Outcomes



Need to plan for contingencies

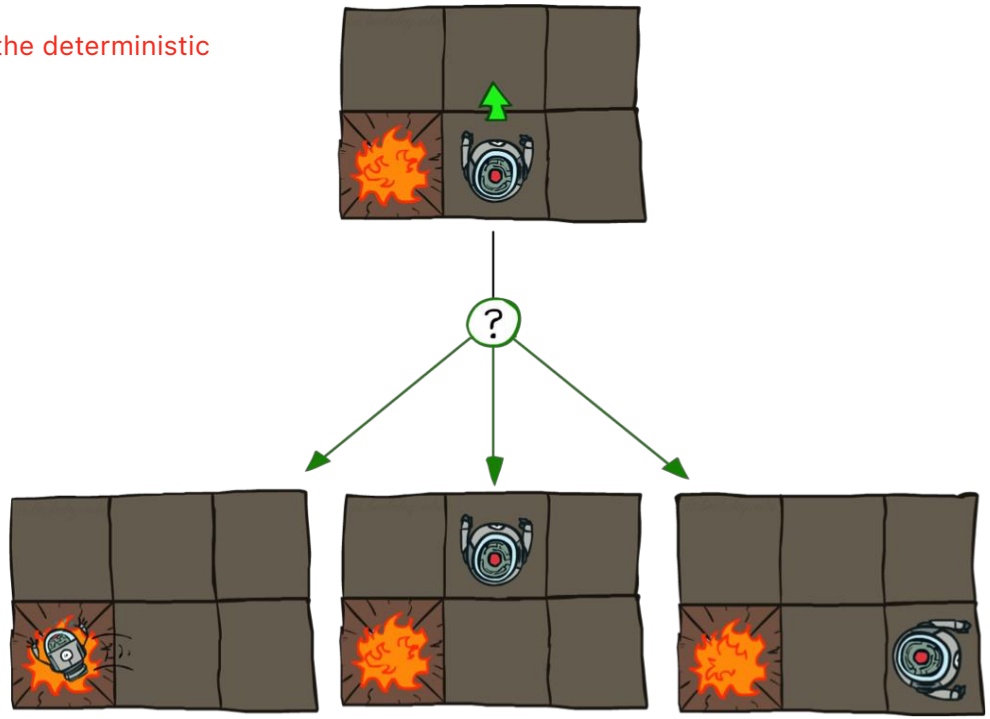
From Exact Plans to Policies (Contingency Plans)

Deterministic Grid World



we need a plan for the deterministic

Stochastic Grid World



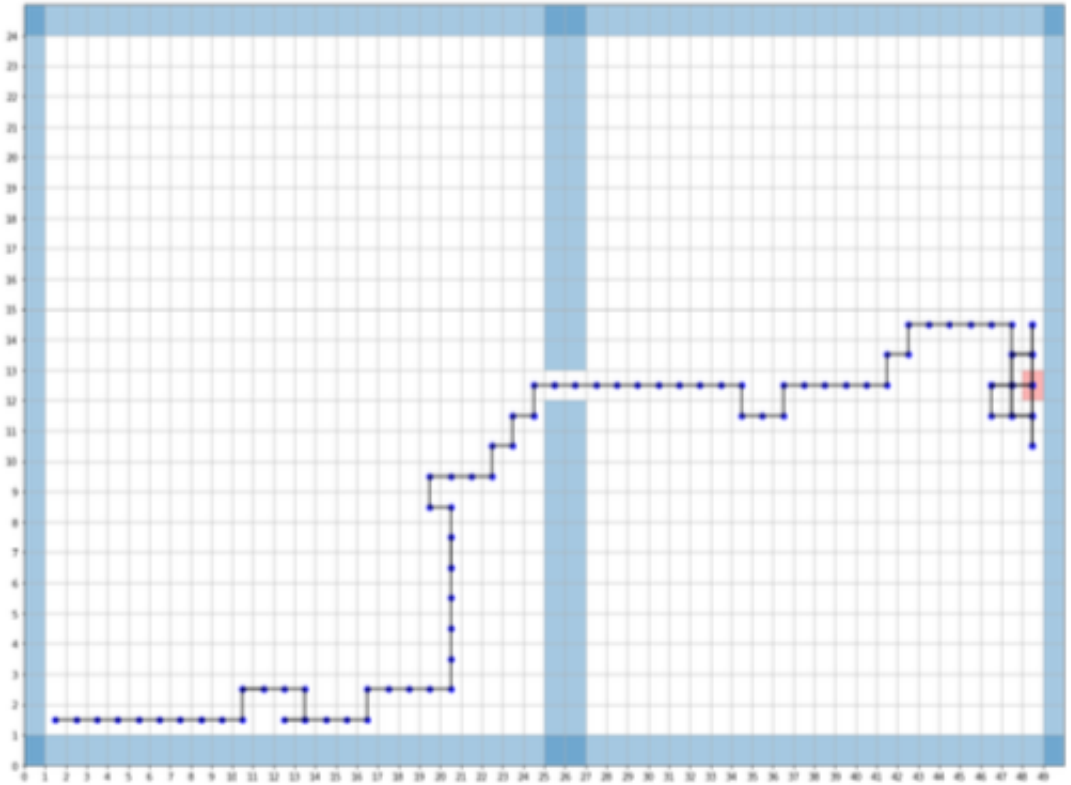
If the environment was *deterministic*, then we would just need an optimal plan (a sequence of actions) from start to the goal. If there is *non-determinism*, we are not sure where we will land up, hence need a policy that prescribes actions from each state.

Example: Multiple paths are possible due to uncertainty

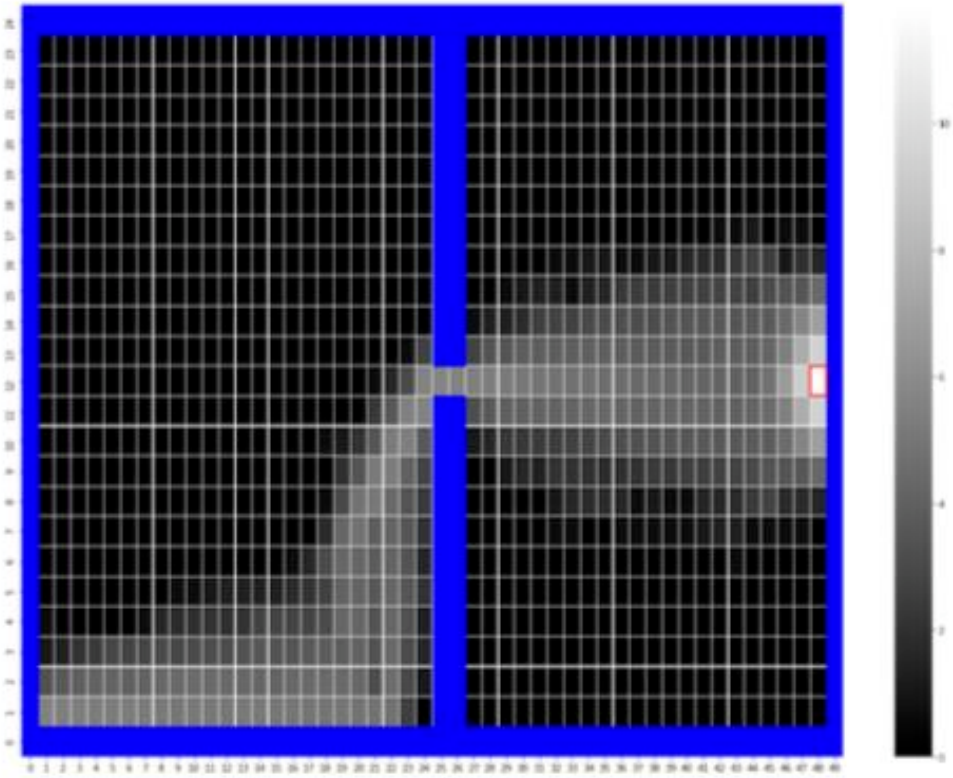
Moving in an MDP is a stochastic process. At every state you look up the action prescribed by the policy and execute it. The effects are uncertain and we can land up in unintended states.

MDP -> markov decision process

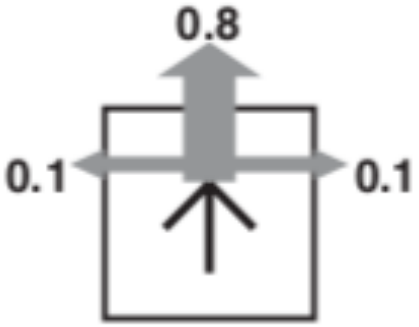
A sample path through the MDP



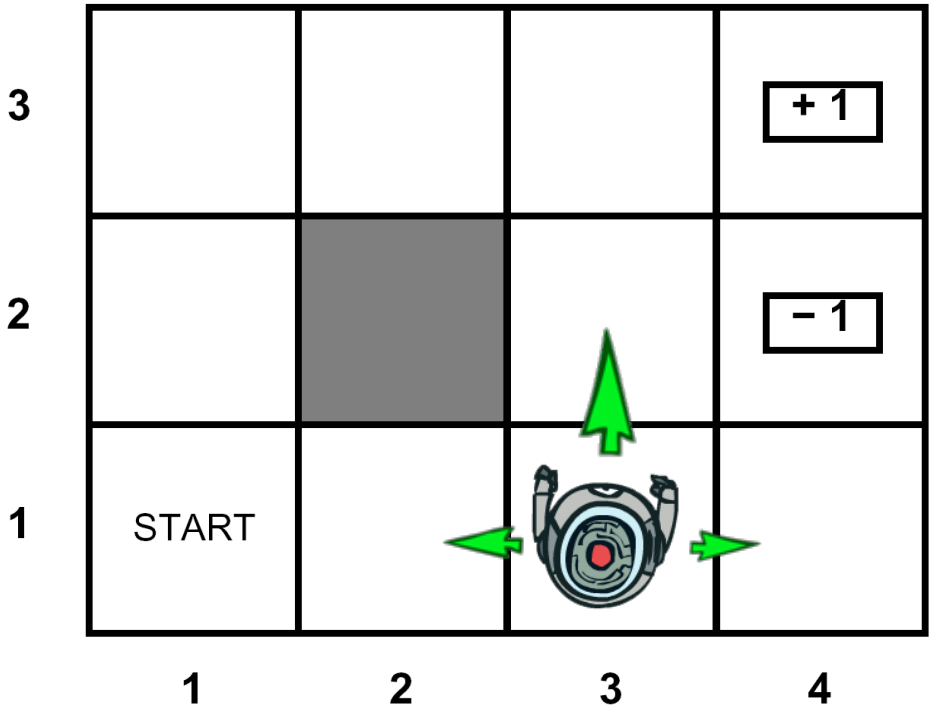
State visitations after multiple runs



Example: Grid World Domain



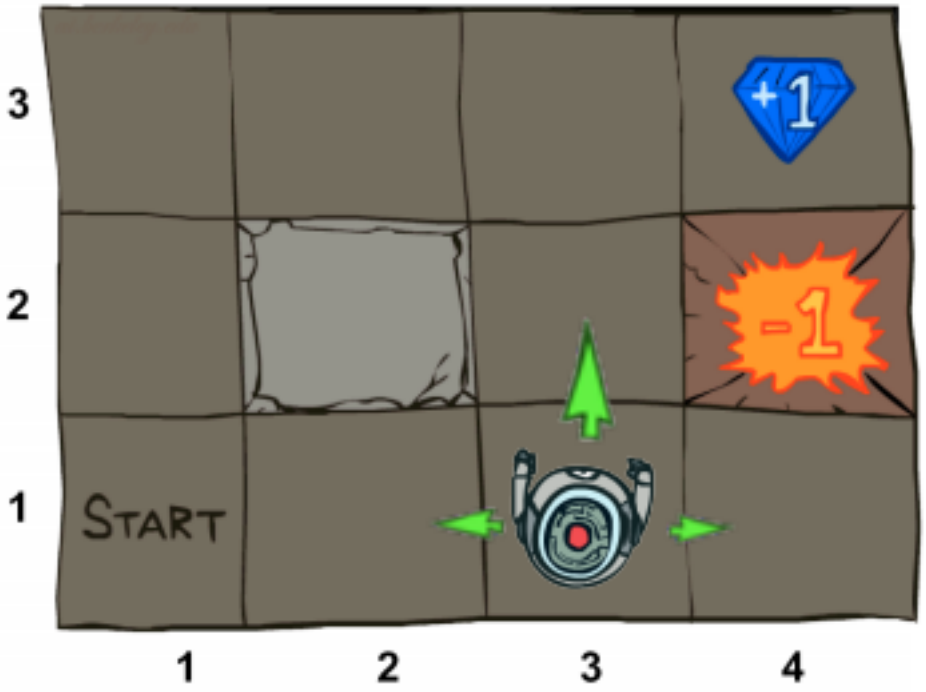
- Actions do not always go as planned
  - 80% of the time, the action North takes the agent North (if there is no wall there)
  - 10% of the time, North takes the agent West; 10% East
  - If there is a wall in the direction the agent would have been taken, the agent stays put
- The agent receives rewards each time step
  - Small “living” reward each step (can be negative)
  - Large rewards come at the end (negative or positive)
- Overall Goal: maximize sum of rewards
  - Fundamentally a sequential decision-making problem.
  - Taking an action now can have an impact later.



Markov Decision Processes – a model to determine how to decide sequential actions when there is uncertainty.

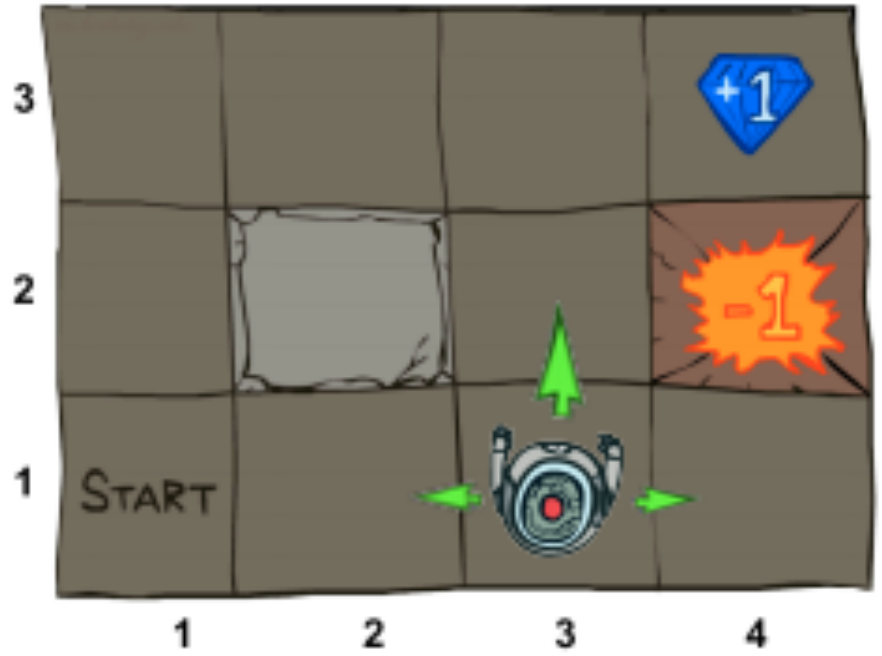
Formally: *Markov Decision Processes (MDPs)*

- An MDP is defined by:
  - A set of states s in S
  - A set of actions a in A
  - A transition function T(s, a, s')
    - Probability that a from s leads to s', i.e., P(s'| s, a)
    - Also called the model or the dynamics
  - A reward function R(s, a, s')
    - Sometimes just R(s) or R(s')
  - A start state
  - Maybe a terminal state



MDPs – Transition Model

- An MDP is defined by:
  - A set of states s in S
  - A set of actions a in A
  - A transition function T(s, a, s')
    - Probability that a from s leads to s', i.e., P(s' | s, a)
    - Also called the model or the dynamics



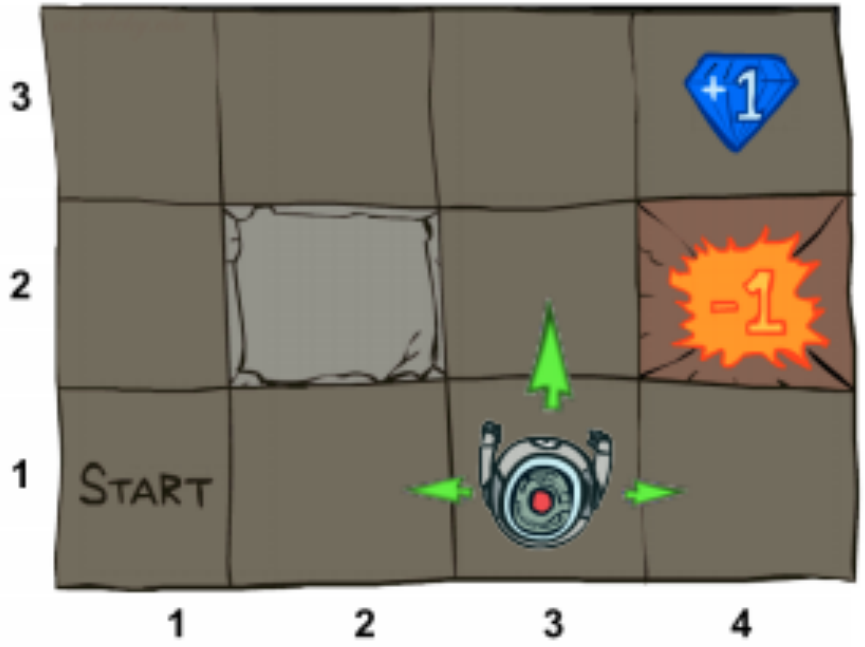
$T(s_{11}, E, \ldots)$
$\ldots$
$T(s_{31}, N, s_{11}) = 0$
$\ldots$
$T(s_{31}, N, s_{32}) = 0.8$
$T(s_{31}, N, s_{21}) = 0.1$
$T(s_{31}, N, s_{41}) = 0.1$
$\ldots$

T is a Big Table!
11 X 4 x 11 = 484 entries

For now, we give this as input to the agent

MDPs – Reward Model

- An MDP is defined by:
  - A set of states s in S
  - A set of actions a in A
  - A transition function T(s, a, s')
    - Probability that a from s leads to s', i.e., P(s' | s, a)
    - Also called the model or the dynamics
  - A reward function R(s, a, s')



$R(s_{32}, \overset{\cdots}{N}, s_{33}) = -0.01$ ← ***Cost of breathing***

$R(s_{32}, \overset{\cdots}{N}, s_{42}) = -1.01$

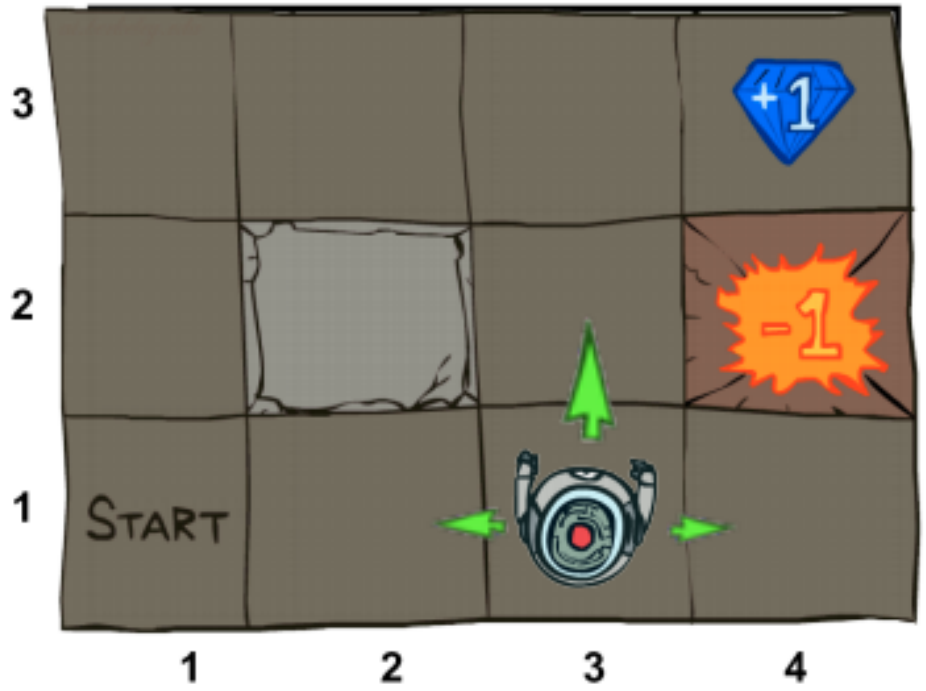
$R(s_{33}, \overset{\cdots}{E}, s_{43}) = 0.99$

R is also a Big Table!

For now, we also give this to the agent

MDPs – Reward Model (Equivalent)

- An MDP is defined by:
  - A set of states s in S
  - A set of actions a in A
  - A transition function T(s, a, s')
    - Probability that a from s leads to s', i.e., P(s'| s, a)
    - Also called the model or the dynamics
  - A reward function R(s, a, s')
    - Sometimes just R(s) or R(s')



...

$R(s_{33}) = -0.01$

$R(s_{42}) = -1.01$

$R(s_{43}) = 0.99$

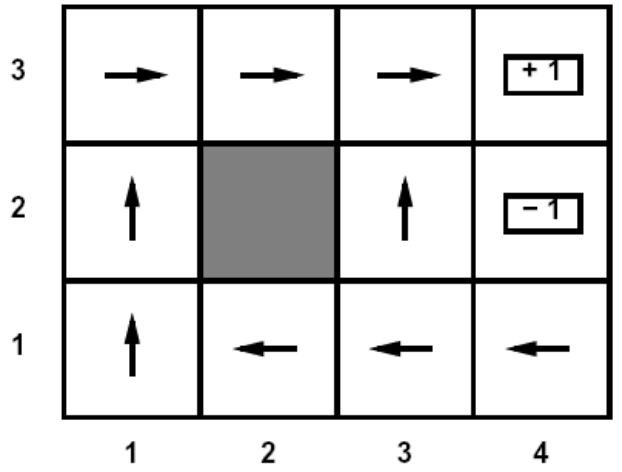
Note: two notations are followed in literature. One in which rewards are associated with states and in the other rewards are associated with state transitions. Both the notations are equivalent and accepted.

Policies: *A map from states to actions*

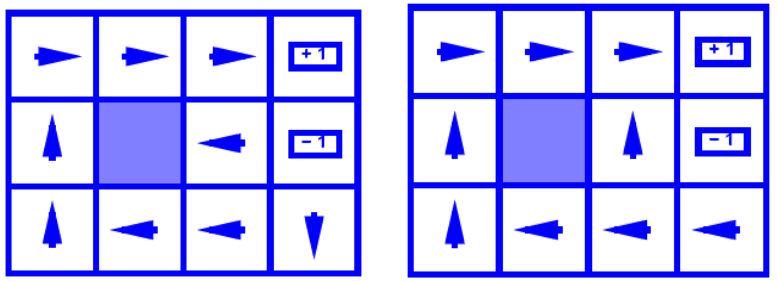
- Deterministic single-agent search problems
  - We determined the optimal plan, or sequence of actions, from start to a goal

- For MDPs, we want an optimal policy $\pi^*: S \rightarrow A$
  - A policy π gives an action for each state
  - **An optimal policy is one that maximizes the expected utility if followed**
  - The agent arrives at a state and looks up the action according to the policy.

- Note: there can be many policies, we are to determine the optimal one.



Optimal policy when R(s, a, s') = -0.03 for all non-terminals s



There can be other policies that prescribe different actions in a state.

MDPs (the agentic view)

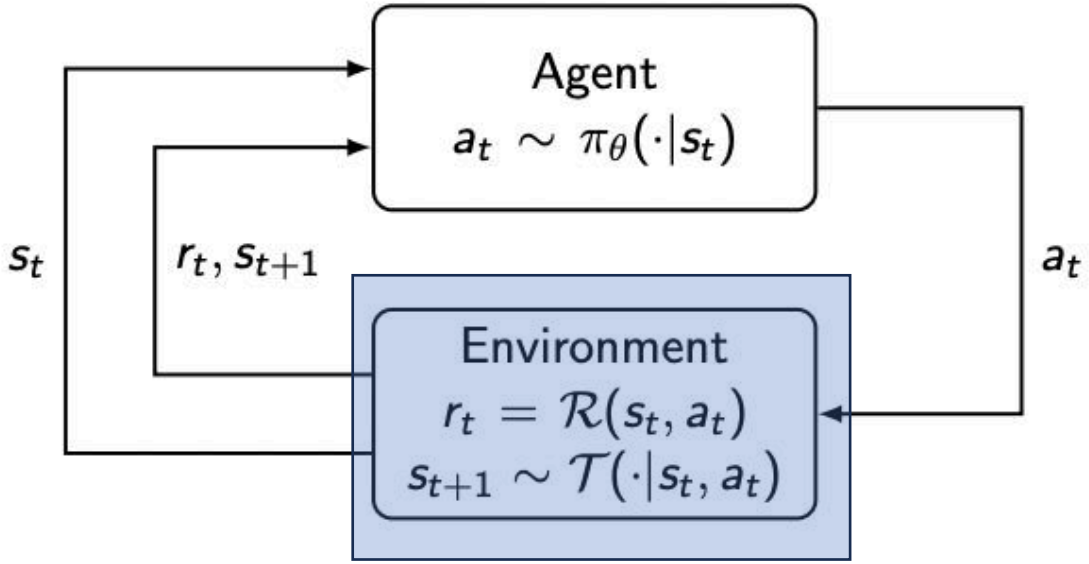
- **Solving an MDP means**
  - Find a policy that maximizes the future expected reward.
  - That is, find a prescription of actions from each state such that the future expected reward is maximized.

- **Agentic View**
  - The AI agent can be considered as running the policy in its mind at each time step.
  - At each transition, there is a cost incurred or equivalently a reward is received.

- **What is the goal of our AI system?**
  - **Goal of the AI agent is to maximize the reward**
  - Single step? Yes of course. But that is myopic.
  - Sequentially? Yes. We want to maximise the expected reward for all times. This is a sequential problem.

Notion of an AI agent.

- We can think of the policy as the **agent** that takes in the state from the environment (or its domain) and prescribes an action.
- The action is executed and its consequences are observed in the next time step.



The environment gives the agent a reward. It also transitions to the next state, not under the agent's control.

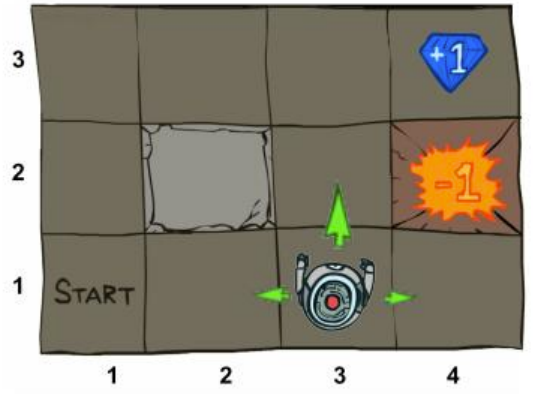
Rewards *reflect* our intended goal/objective for the AI system

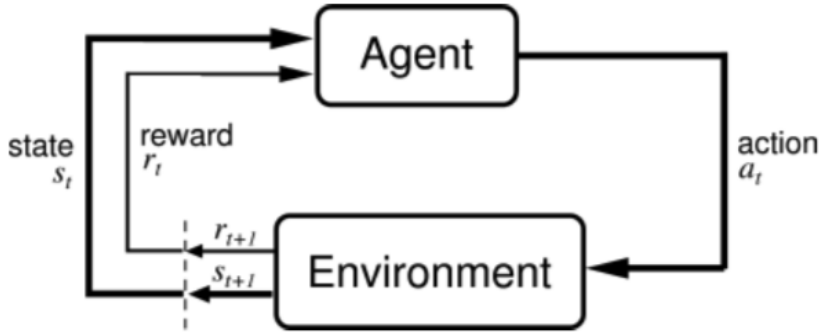
Notion is called the reward hypothesis

- Rewards are scalar values provided by the environment to the agent that indicate ***whether goals have been achieved***, e.g., 1 if goal is achieved, 0 otherwise, or -1 for overtime step the goal is not achieved

- Goals specify ***what*** the agent needs to achieve, not how to achieve it.

- Goal seeking behaviour, achieving purposes and expectations can be formulated mathematically as ***maximizing expected cumulative sum of scalar values.***

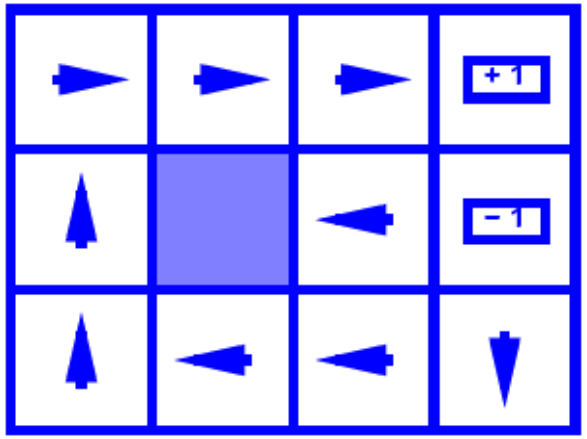




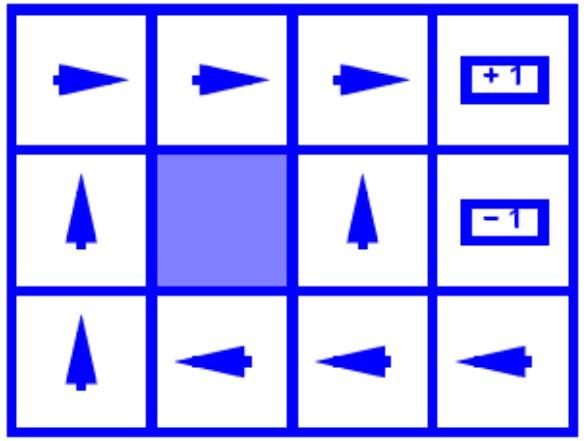
$$\pi^* = \arg\max_{\pi} \mathrm{E}[\sum_{t=0}^{H} \gamma^t R_t(S_t, A_t, S_{t+1})|\pi]$$

How good (optimal) behaviour changes with reward modulation?

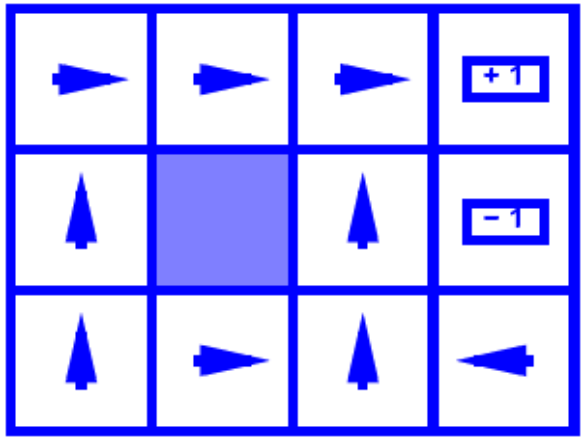
- Interpret reward as the cost of breathing (living reward).
- The value of R(s) balances the *risk* and *reward* that the agent takes.



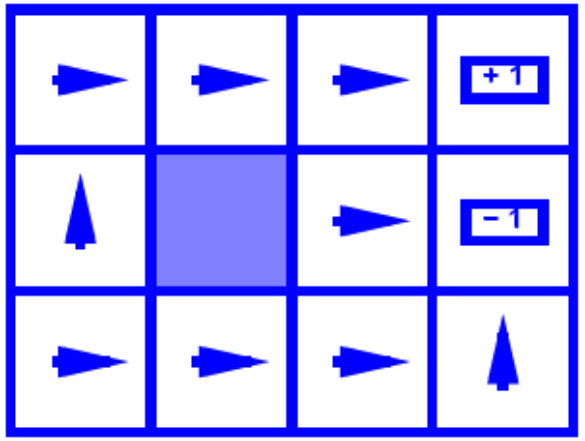
R(s) = -0.01



R(s) = -0.03



R(s) = -0.4



R(s) = -2.0

Reducing the living reward.
Analysing the impact on the optimal policy.

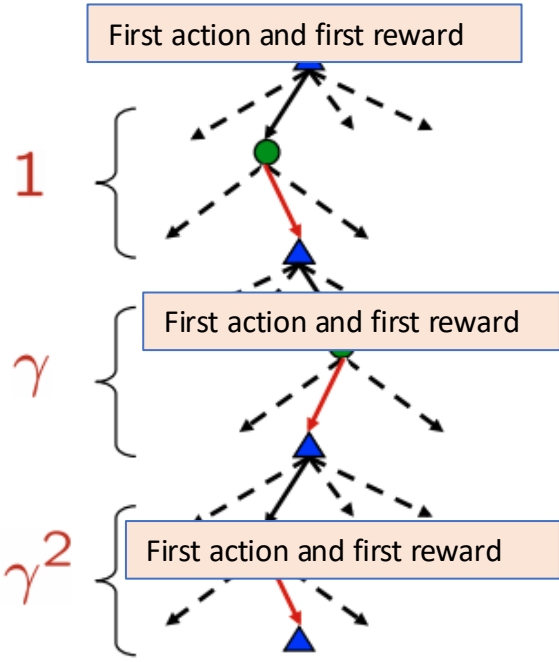
We seek to measure reward sequences

- How to measure the utility (goodness) or a reward sequences.
- Note the agent is interacting sequentially in the domain – we have a sequence of rewards.

What preferences should an agent have over reward sequences?

- More or less? [1, 2, 2] or [2, 3, 4]
- Now or later? [0, 0, 1] or [1, 0, 0]

- How to discount?
  - Each time we descend a level, we multiply in the discount once
- Why discount?
  - Sooner rewards probably do have higher utility than later rewards
  - Also helps our algorithms converge
- Example: discount of 0.5
  - U([1,2,3]) = 1*1 + 0.5*2 + 0.25*3
  - U([1,2,3]) < U([3,2,1])



Principle:

- Maximize the sum of rewards
- Prefer rewards now to rewards later - discounting

[Some evidence from economics/behavioural sciences that this is true]

Assigning Utilities to Reward Sequences

- Additive utility: $U([r_0, r_1, r_2, \ldots]) = r_0 + r_1 + r_2 + \cdots$
- Discounted utility: $U([r_0, r_1, r_2, \ldots]) = r_0 + \gamma r_1 + \gamma^2 r_2 \cdots$

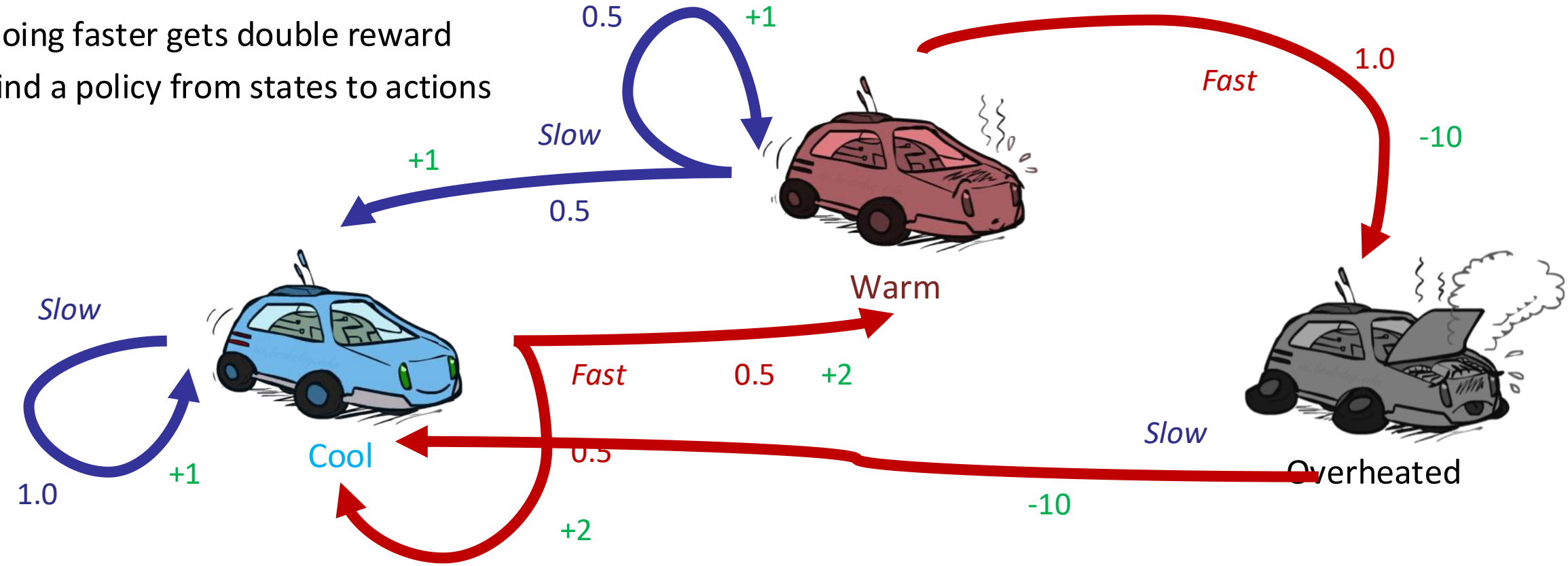
Discounting appears to be a good model for both animal and human preferences over time. Computationally, helps us converge utilities of infinite sequences.

With discounted rewards, the utility of an infinite sequence is finite.

$$U([r_0, \ldots r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq R_{\max}/(1-\gamma)$$

Example: Racing Car

- A robot car wants to travel far, quickly
- Three states: Cool, Warm, Overheated
- Two actions: *Slow*, *Fast*
- Going faster gets double reward
- Find a policy from states to actions



How to compute the “optimal” policy?

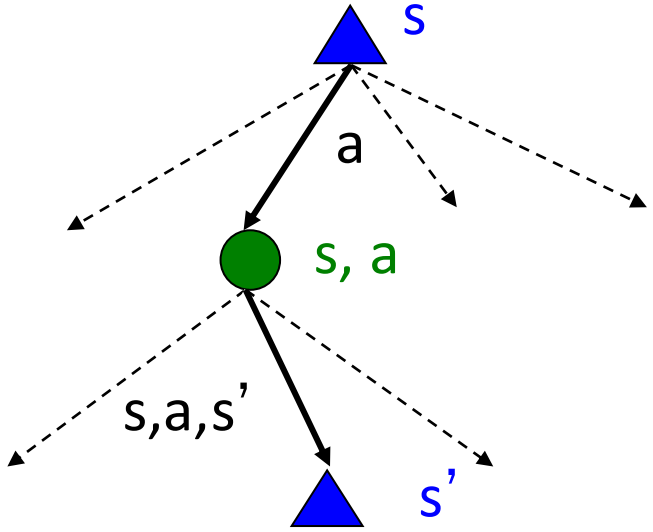
One way is policy iteration followed by policy extraction

MDPs – *A closer look at state-action-state transition*

- **Markov decision processes:**
  - Set of states S
  - Start state s_0
  - Set of actions A
  - Transitions P(s'|s,a) (or T(s,a,s'))
  - Rewards R(s,a,s') (and discount γ)

- **MDP quantities:**
  - Policy = Choice of action for each state
  - Utility = sum of (discounted) rewards

- ***Finally our goal is to solve solve the MDP?***
  - *How to determine the optimal policy?*

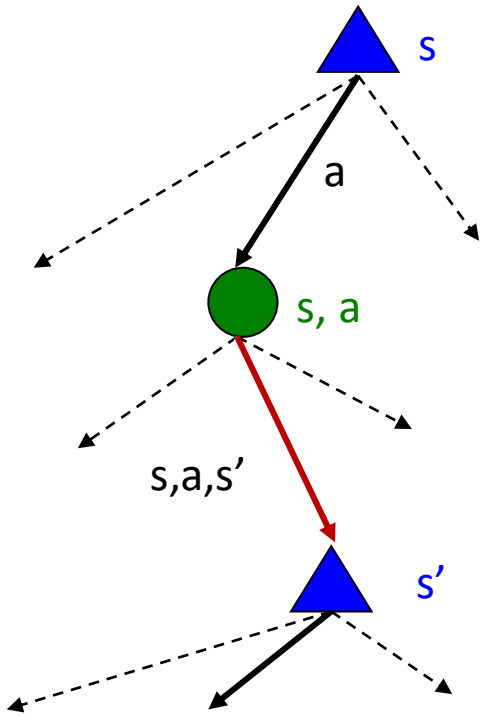


Notation that we will need

- **The optimal policy:**
  $\pi^*(s)$ = optimal action from state s

- **The value (utility) of a state s:**
  $V^*(s)$ = expected utility starting in s and acting optimally

- **The value (utility) of a q-state (s,a):**
  $Q^*(s,a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally. Essentially, what is the best we can do with action a taken on state s.



s is a *state*

(s, a) is a *q-state*

(s,a,s') is a *transition*

Value Function

- Because we want to find a policy that maximizes the expected return, it is a good idea to **estimate the expected return.**

- Value functions represent the **expected return**, for every state.
  - Value functions capture the knowledge of the agent regarding how good is each state for the goal that it is trying to achieve.
  - It is a map where you put in a state and it gives the value of the goodness. It is like the heuristic function.

- Note: there can be many value functions! They take state in and produce a value.

Value Function: V(s)

- **Optimal** value functions measure the **best possible goodness** of states or state/action pairs under all possible policies.

- We can derive the optimal policy from the optimal value function.
- Note: value functions are also called utility functions.

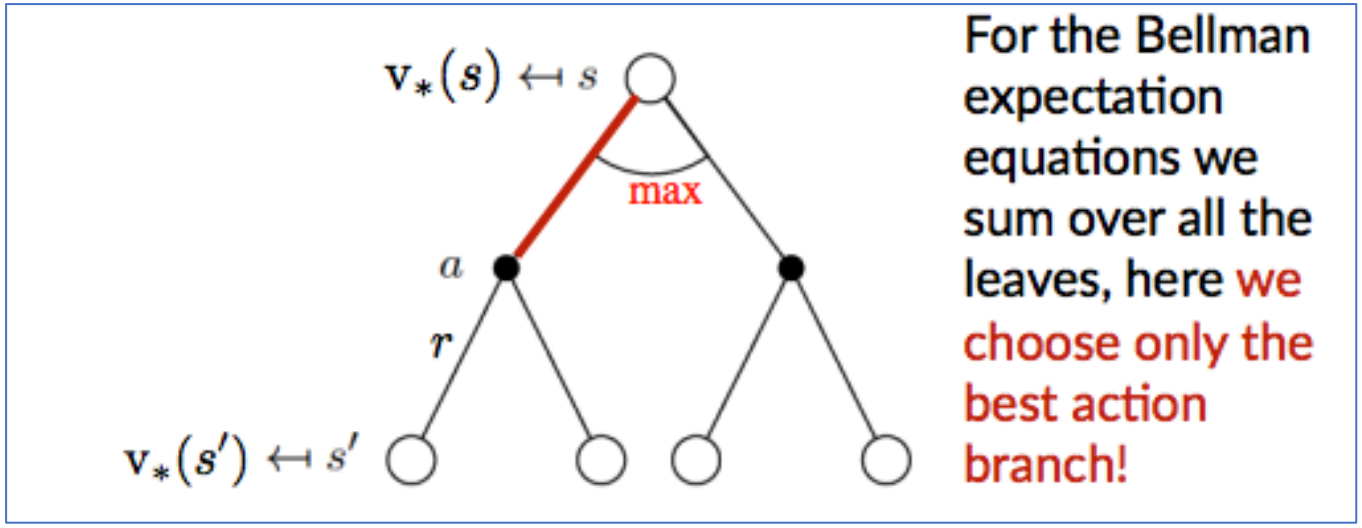


Next, we examine how to compute the value function?

Bellman Equations


(1920-1984)

The value of a state under an **optimal** policy must equal the expected return for the **best** action from that state



What is the intuition?

- Value of a state is not arbitrary, it is influenced by the neighbors.
- The Bellman equation encode this relationship.
- Methods to compute the policy exploit this relationship.

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Value Iteration Computes the Optimal Value Function by using Bellman Equations

- Bellman equations **characterize** the optimal values:

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

- Value iteration **computes** them:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s')\right]$$

- Value iteration is a **fixed-point solution** method

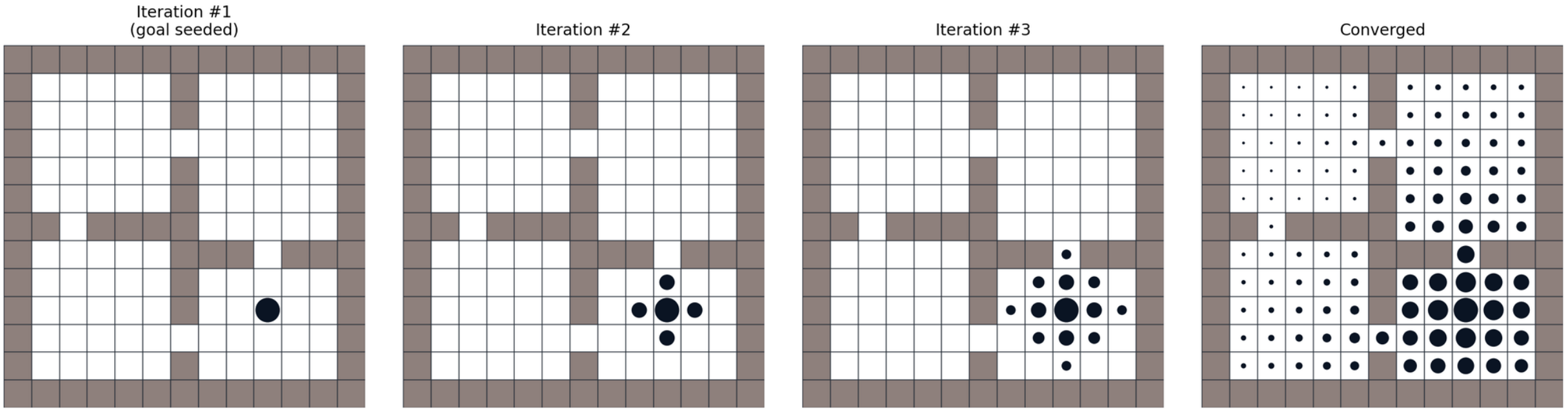
Core Question: How to find the optimal value function?
Exploit the recursive structure. Invoke dynamic programming.

Illustration: Rooms Example

70% Success Rate: When the agent chooses an action (for example, “Move North”), the action succeeds and moves the agent in the intended direction 70% of the time.
30% Failure Rate: 30% of the time, the action fails. Depending on the exact transition model of the GridWorld, a failure usually means the agent slips, stays in place, or drifts sideways relative to the intended command.

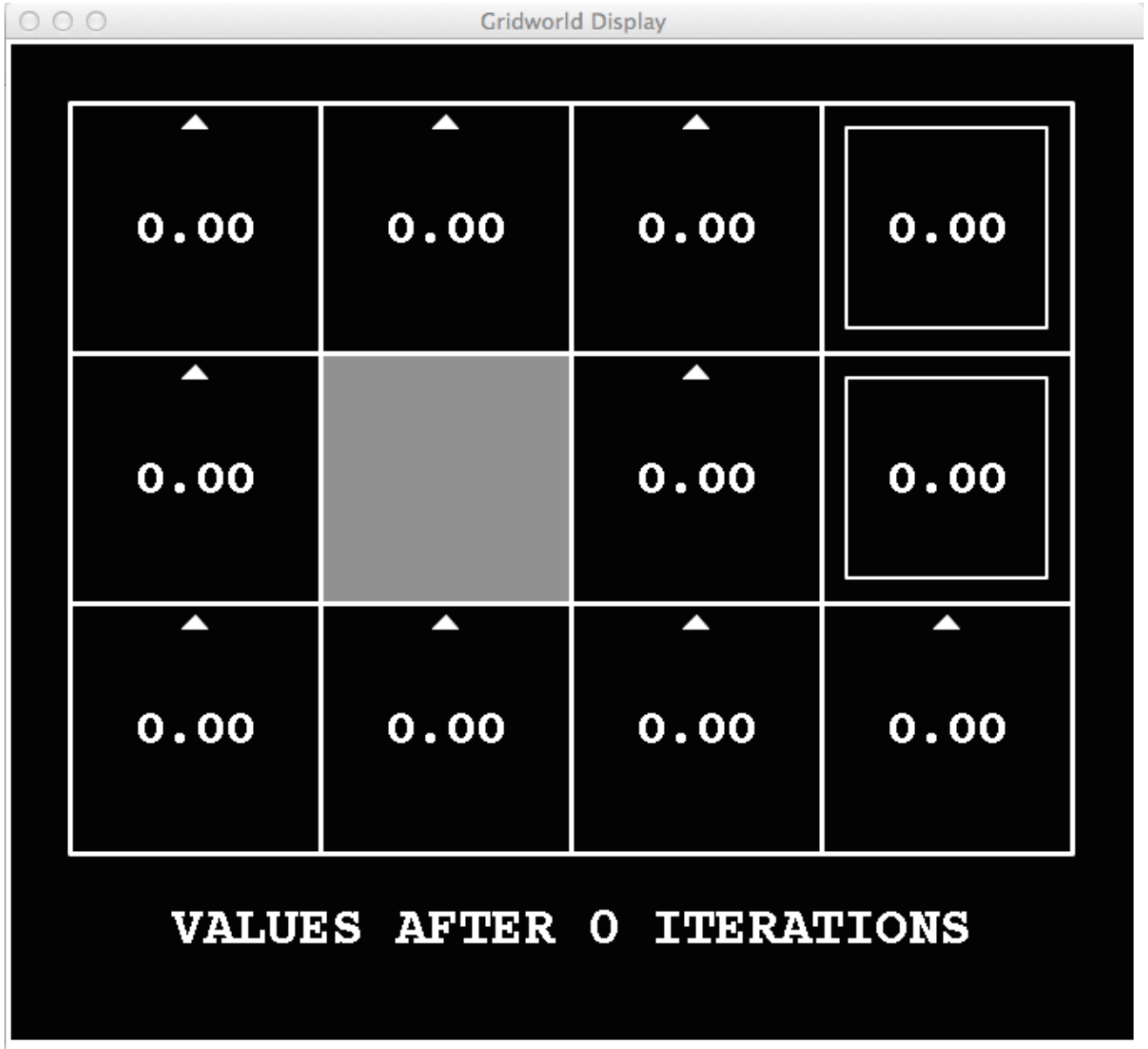
- Each square is a state; black squares are walls, initial circle (left) is the goal state
- Four actions, fail 30% of the time
- No rewards until the goal is reached, $\gamma = 0.9$.
- Circles indicate the magnitude of the value of the corresponding state (no circle means 0 value)
- Values propagate backwards from the goal

this is an example of the sparse reward structure



Example of Value Iteration on Grid World Domain

k=0



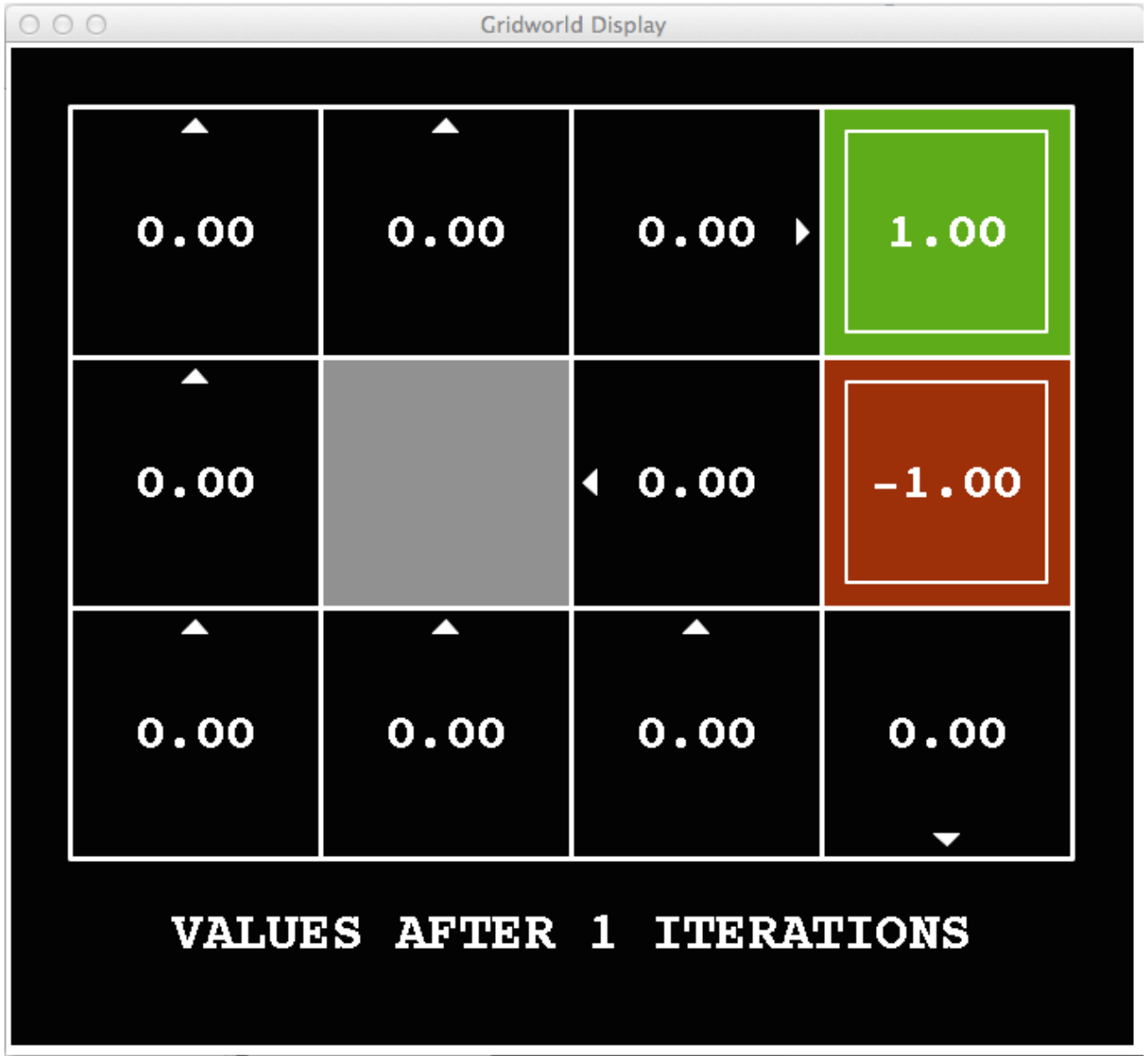
noise = 0.2

means 80% we pass and 20% we die

Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

k=1



In the first iteration the terminal states reflect the reward.

Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

k=2

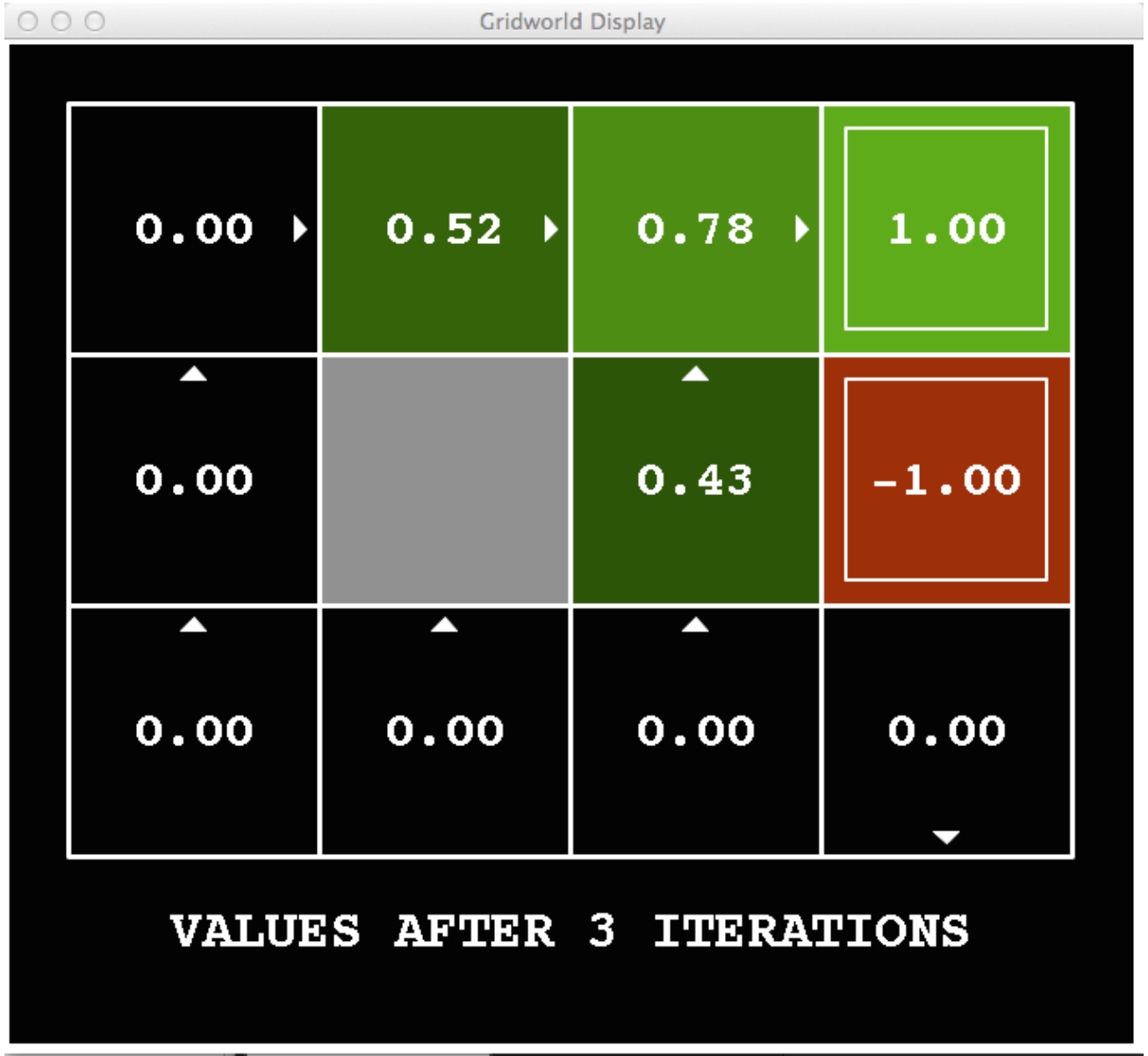


Adjacent states start to get updated.

Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

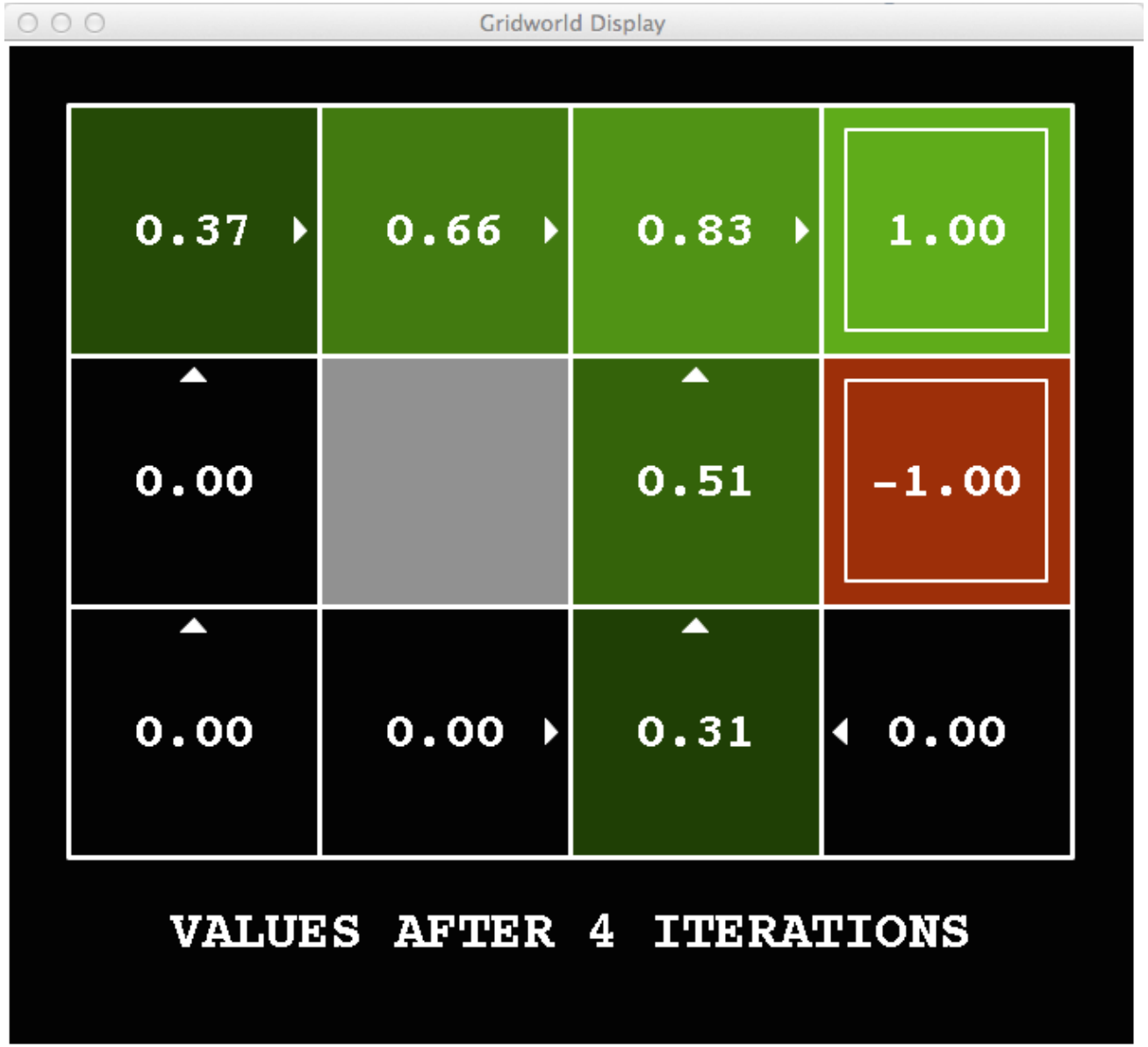
k=3



Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

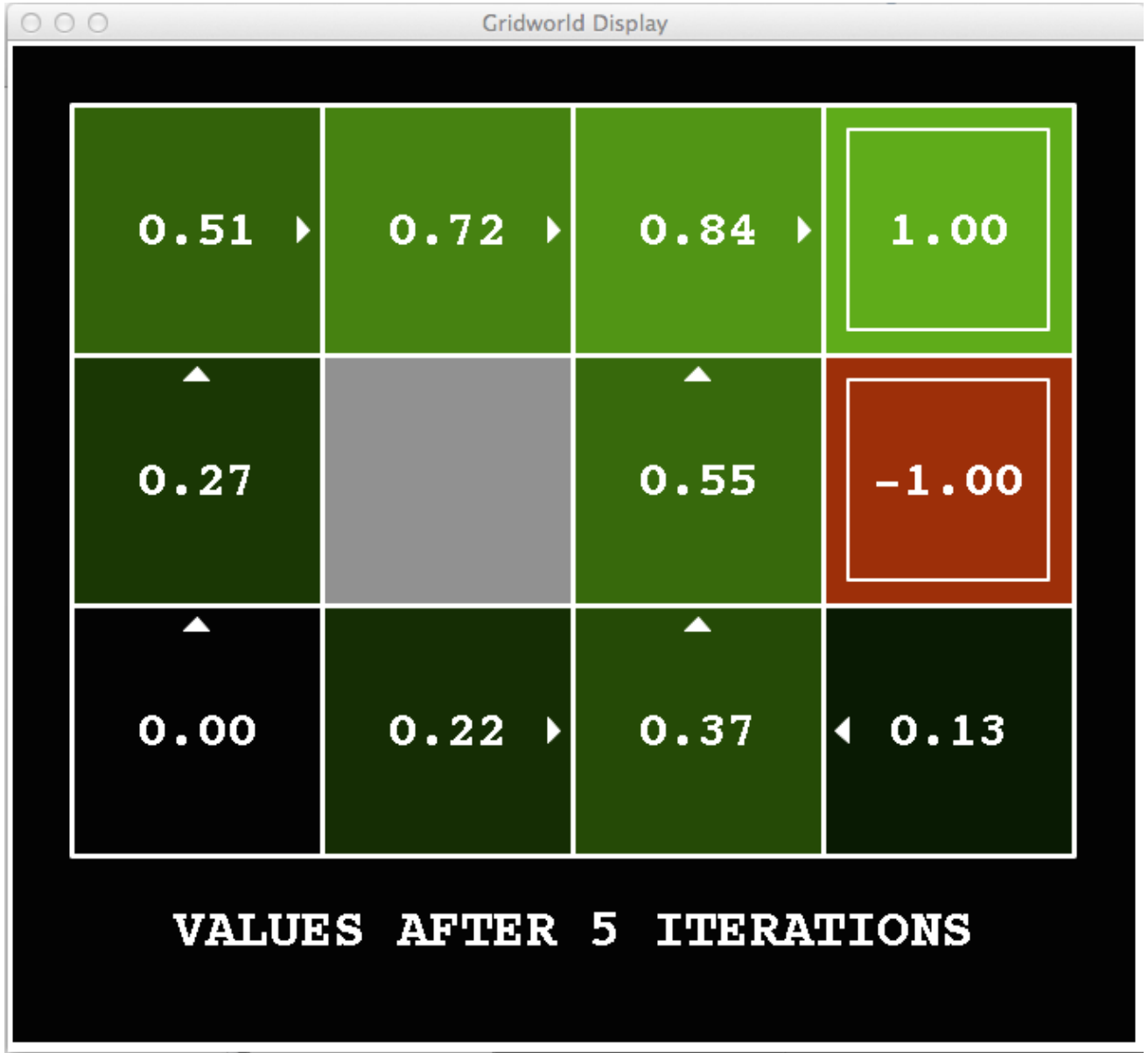
k=4



Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

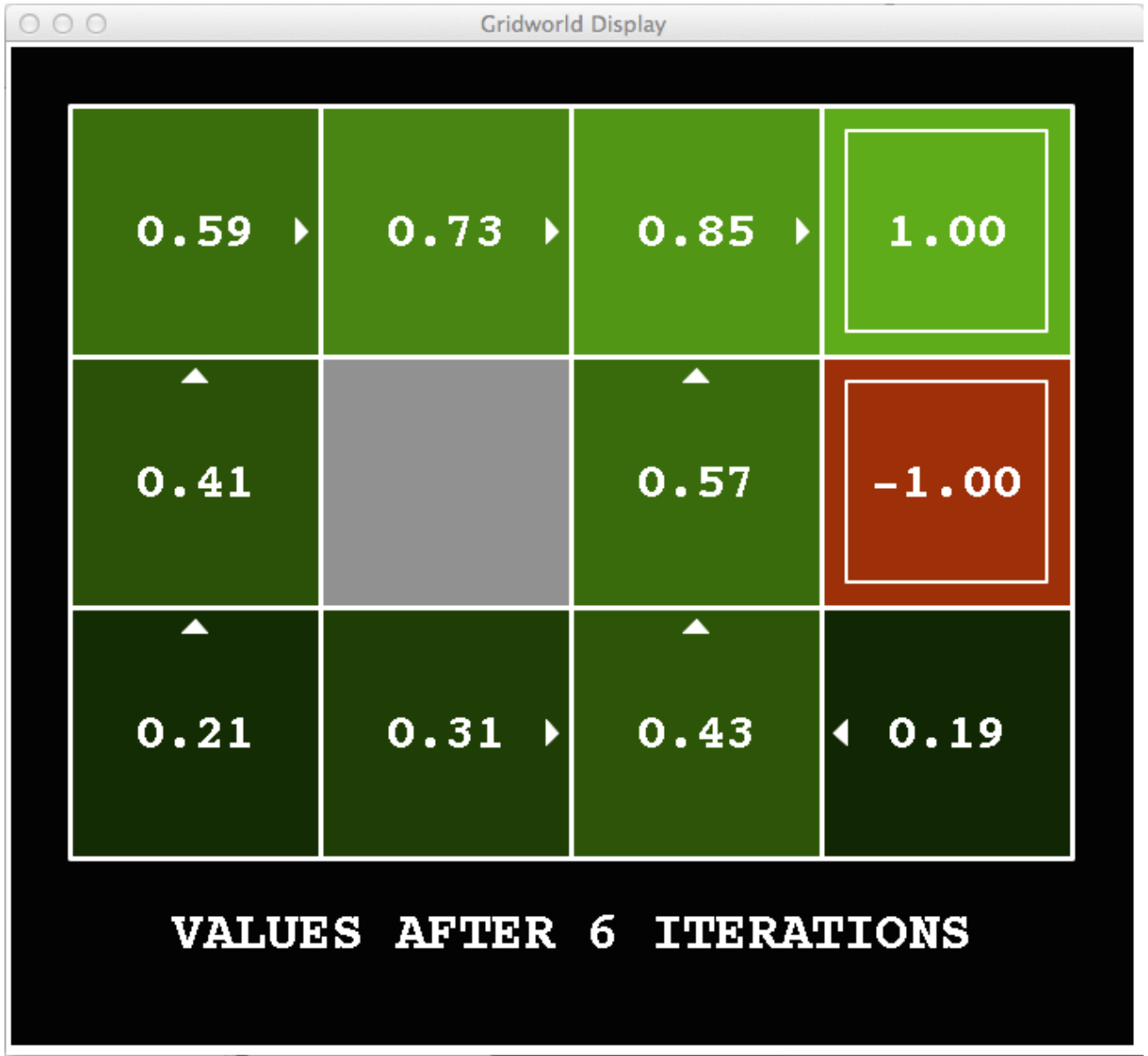
k=5



Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

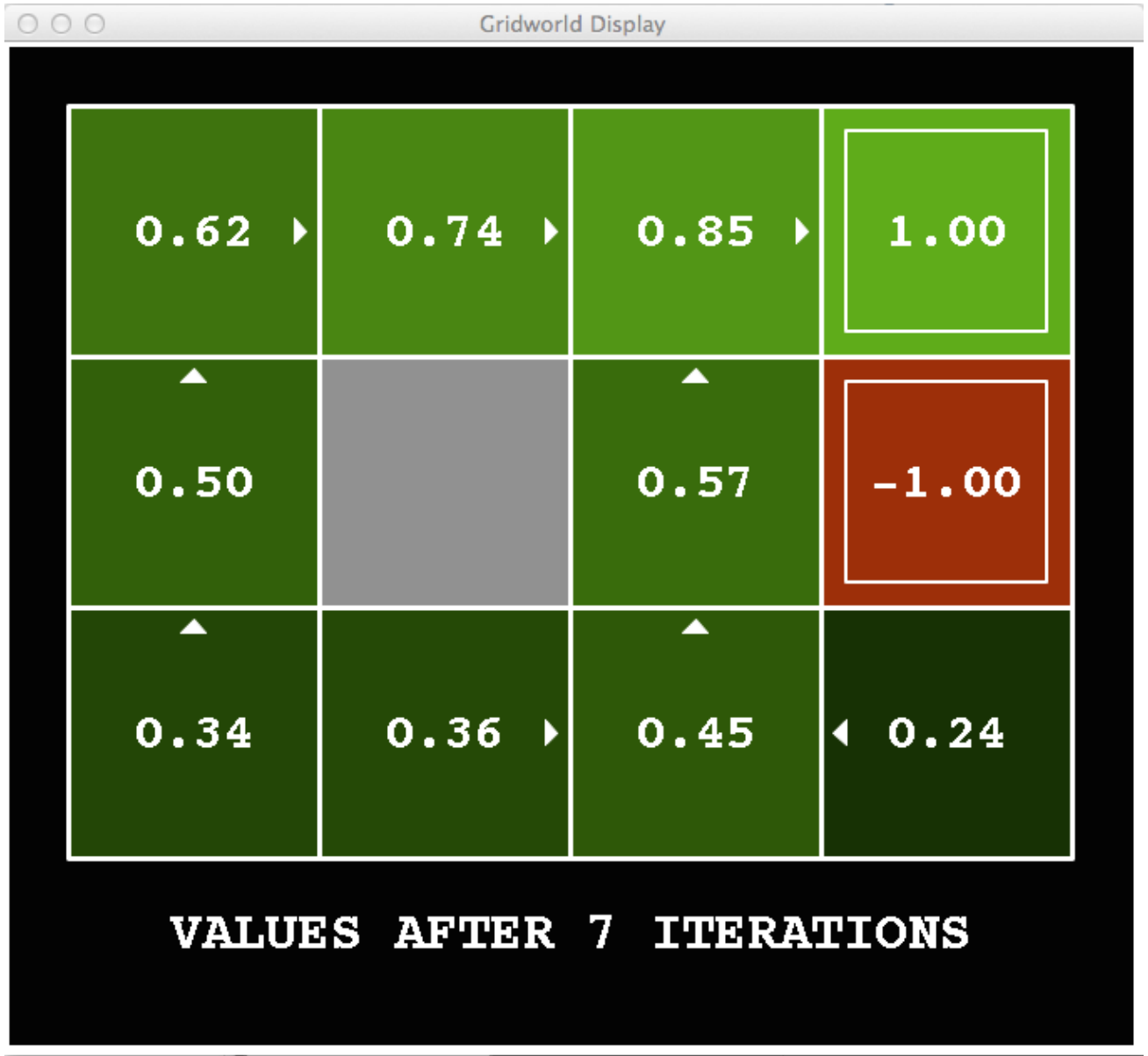
k=6



Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

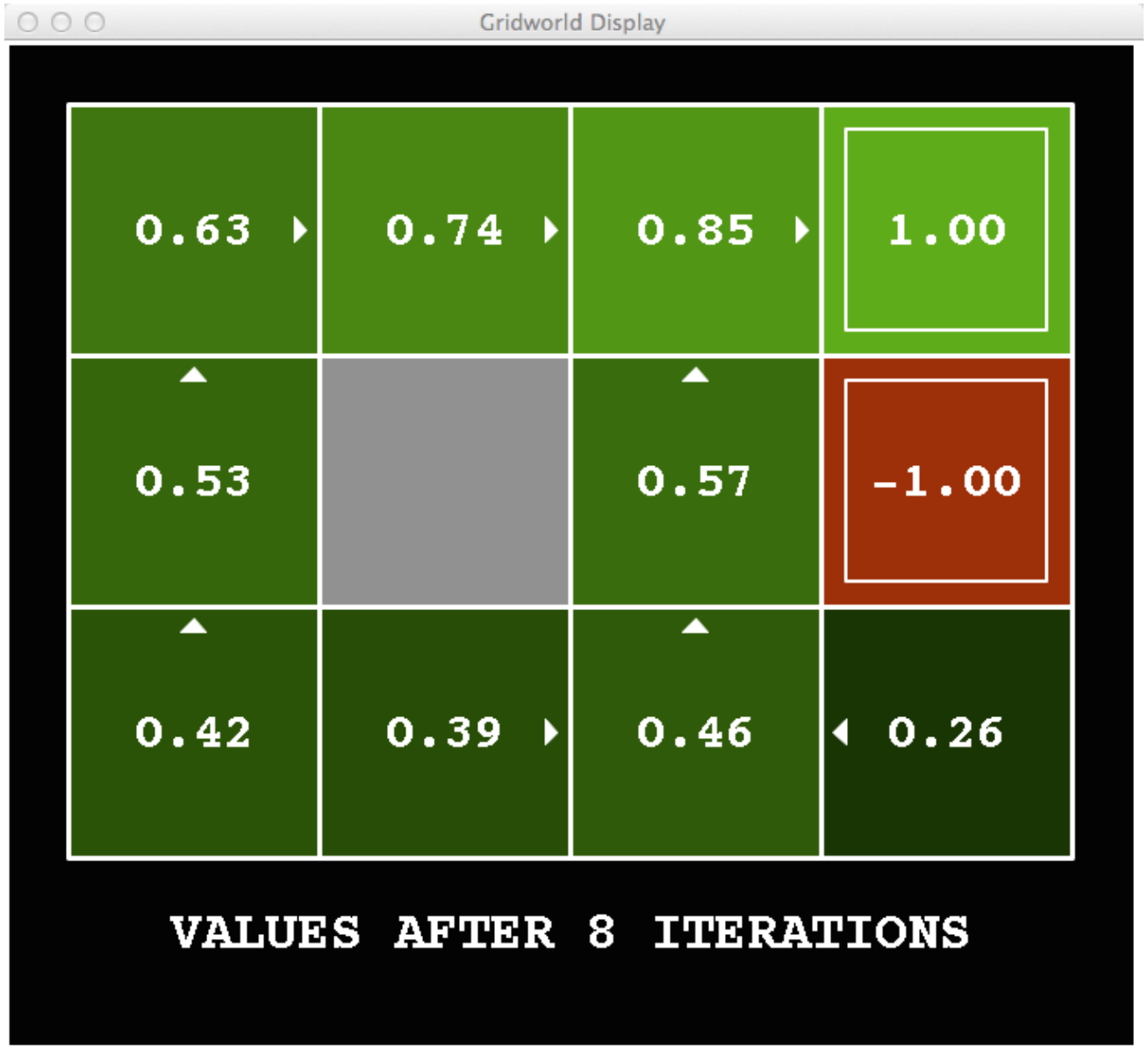
k=7



Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

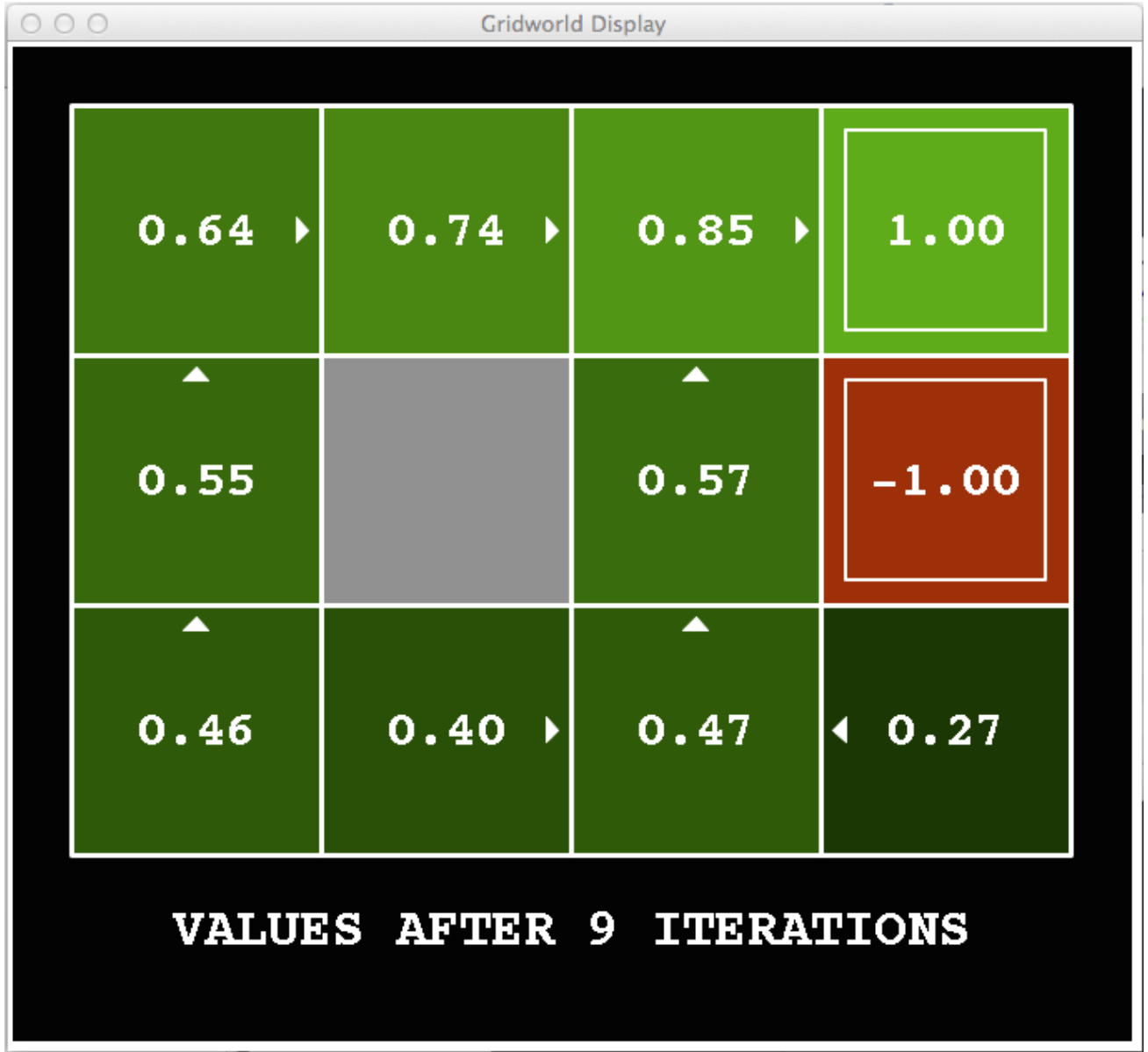
k=8



Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

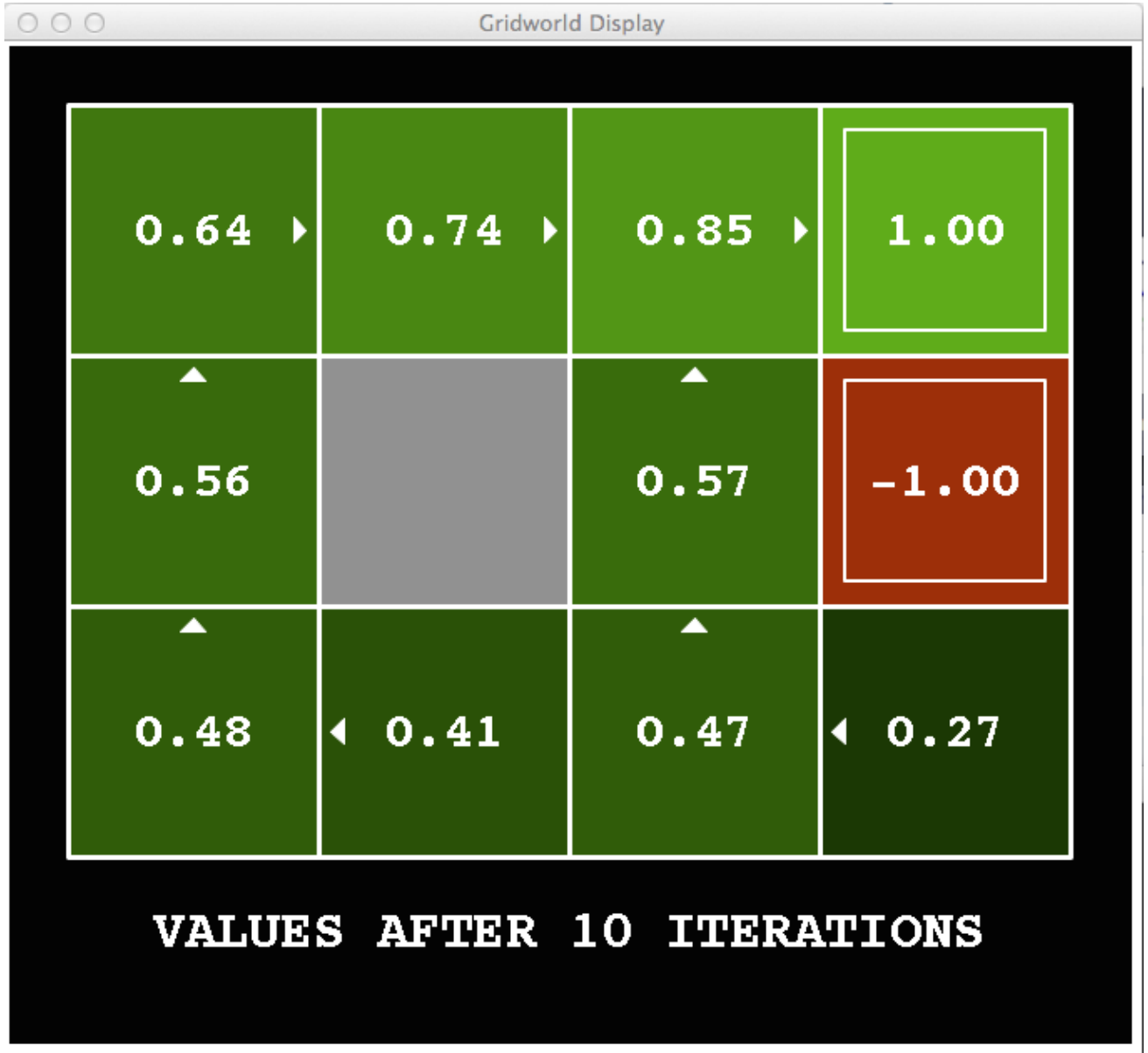
k=9



Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

k=10



Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

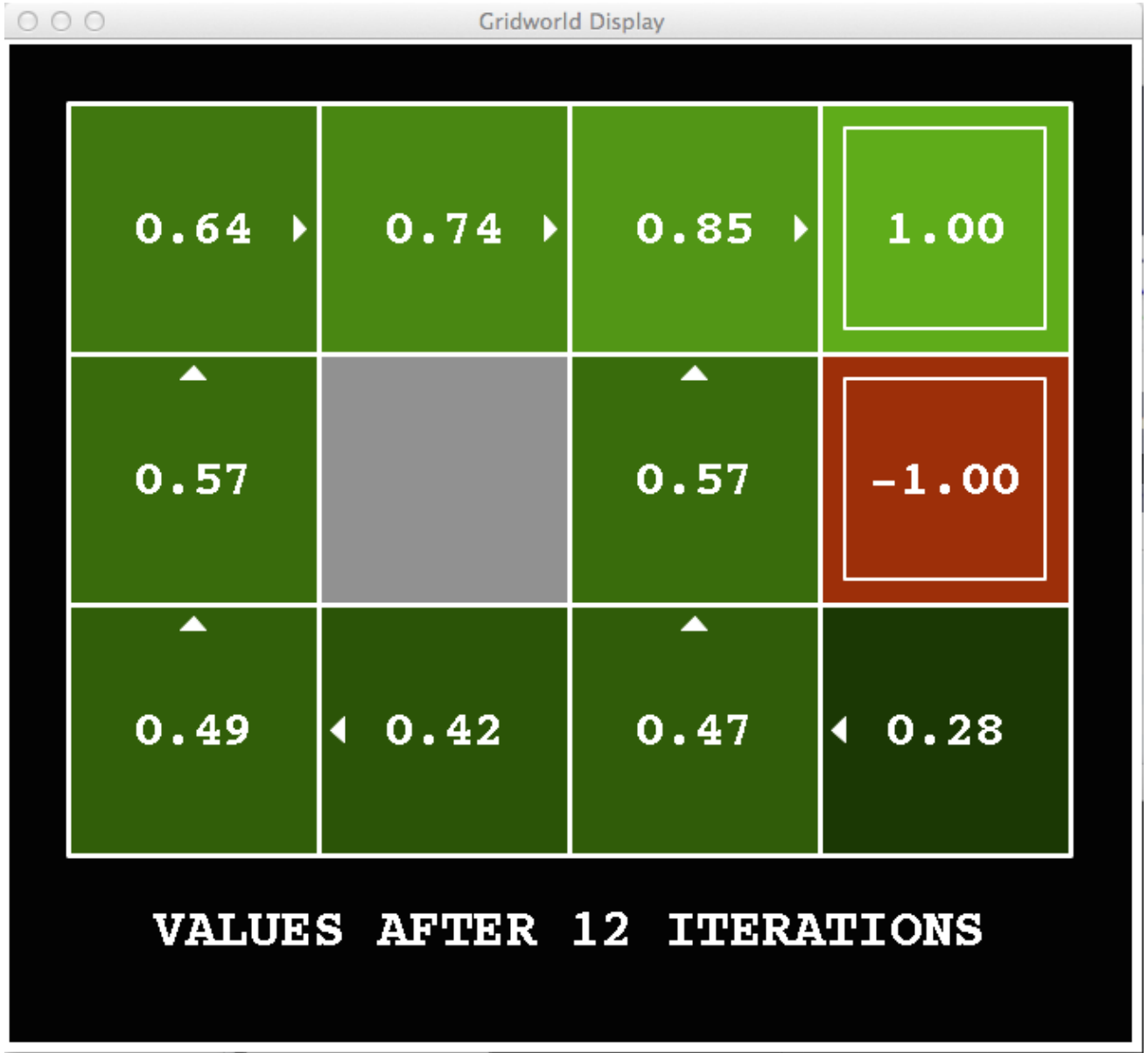
k=11



Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

k=12

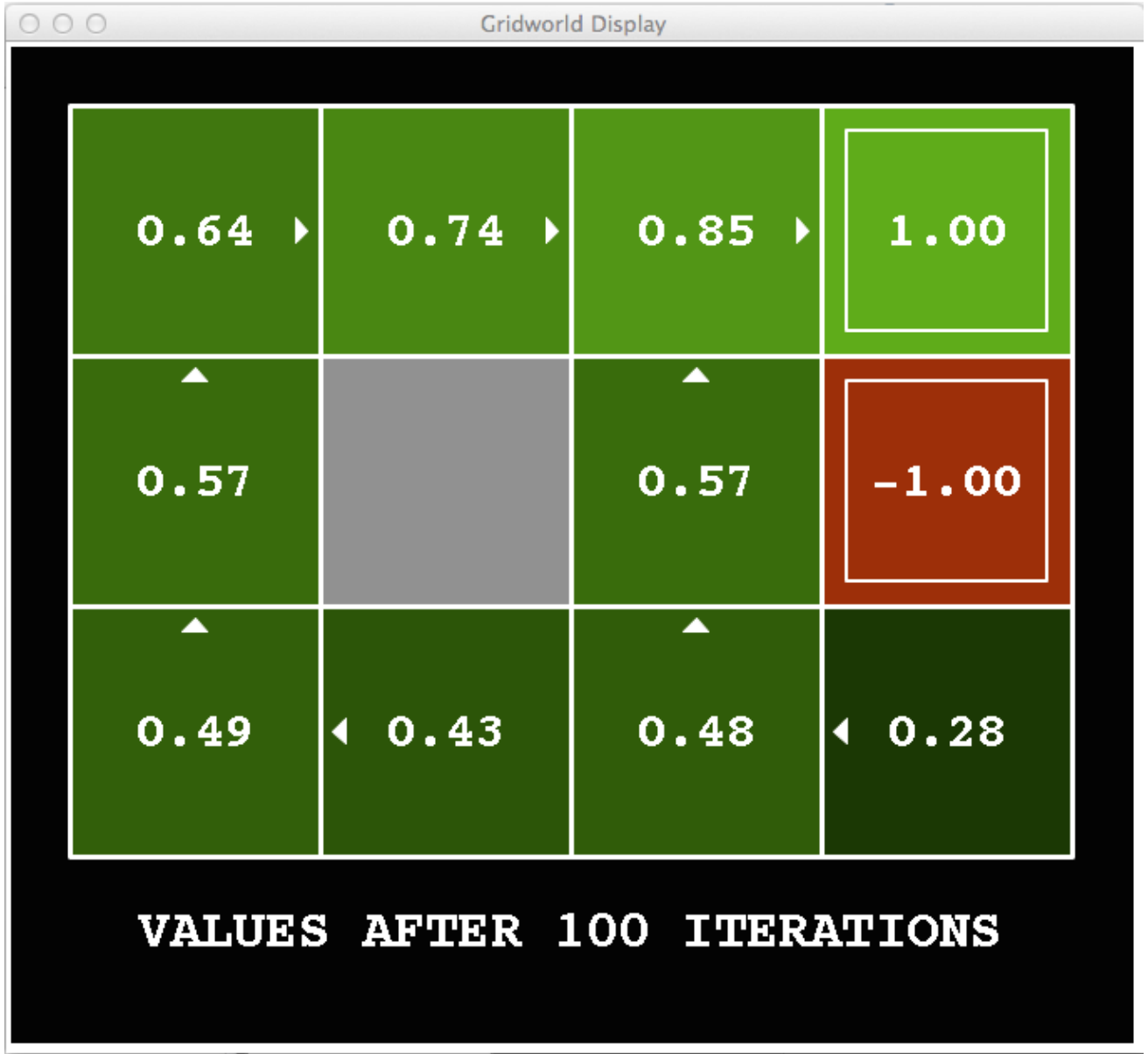


Noise = 0.2
Discount = 0.9
Living reward = 0

Example of Value Iteration on Grid World Domain

k=100

Upon convergence



Noise = 0.2
Discount = 0.9
Living reward = 0

the π represent the policy in the reinforcement learning an the Markov Decision Proces

Value Iteration: Pseudo-code

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$

Sutton and Barto.

Value Iteration: Stopping Criteria

Assuming and infinite horizon MDP with a discount factor less then a relationship between successive difference between value functions and w.r.t to the optimal can be shown.

The relationship leads to a stopping criteria for value iteration.

if $||U_{i+1} - U_i|| < \epsilon(1-\gamma)/\gamma$ then $||U_{i+1} - U|| < \epsilon$

function VALUE-ITERATION(mdp, ϵ) **returns** a utility function
  inputs: mdp, an MDP with states S, actions $A(s)$, transition model $P(s' \mid s, a)$,
    rewards $R(s)$, discount γ
    ϵ, the maximum error allowed in the utility of any state
  local variables: U, U', vectors of utilities for states in S, initially zero
    δ, the maximum change in the utility of any state in an iteration

  repeat
    $U \leftarrow U'$; $\delta \leftarrow 0$
    for each state s **in** S **do**
      $U'[s] \leftarrow R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' \mid s, a)\, U[s']$
      if $|U'[s] - U[s]| > \delta$ **then** $\delta \leftarrow |U'[s] - U[s]|$
  until $\delta < \epsilon(1-\gamma)/\gamma$
  return U

AIMA Ch. 17. Sec. 17.2

What is really going on in value iteration?

- Core idea: the Bellman equations establish a recursive relationship between the values for different states.
- Value iteration is fundamentally doing dynamic programming.
- This is a bootstrapping algorithm.





Value Function and Q-function

Q value function: how can we express it? Take the action a on state s and then act optimally.

$$Q^*(s,a) = \sum_{s'} T(s,a,s')\left[\, R(s,a,s') + \gamma\, V^*(s') \,\right]$$

Optimal value function for a state s maximizes the Q-values of the state s and applicable actions a.

$$V^*(s) = \max_a Q^*(s,a)$$

Recursive way to write the value function (we have seen this definition before).

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma V^*(s')\right]$$

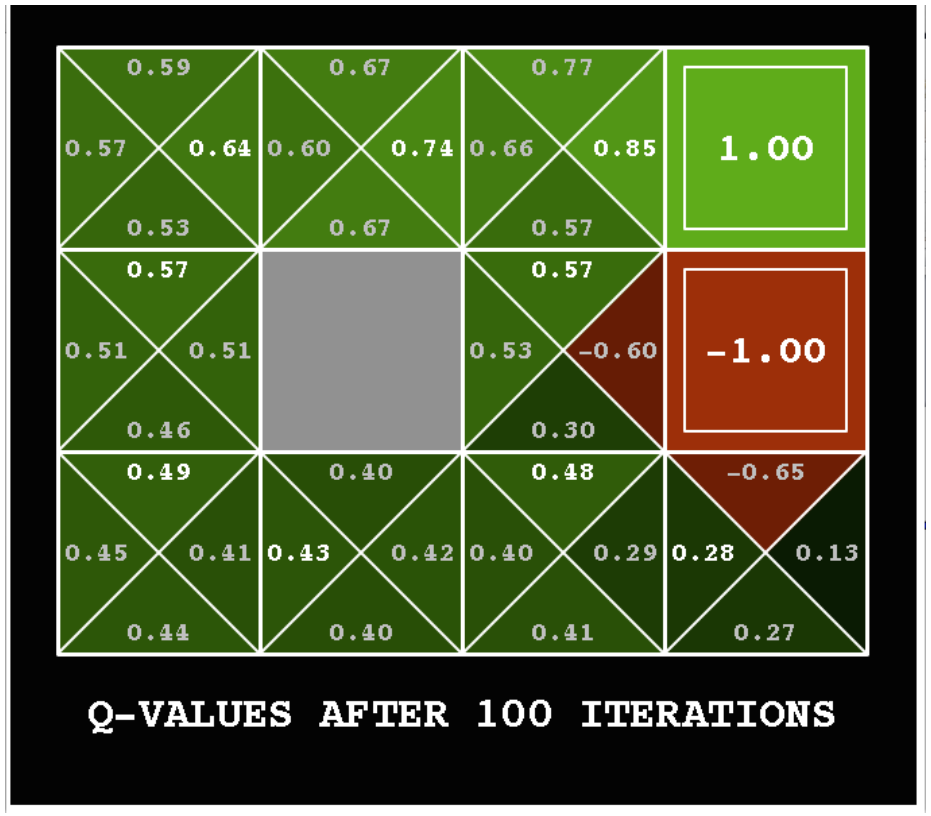
- The utility of a state is the **immediate reward** for that state plus the **expected discounted utility of the next state** assuming that the agent is acting **optimally**.
- Note: The value function has some structure as they are modeling max. rewards that one can obtain from that state. The Bellman equations express that connection.

Example: Optimal Value and Q-functions

Value (utility) of states V(s) for all states



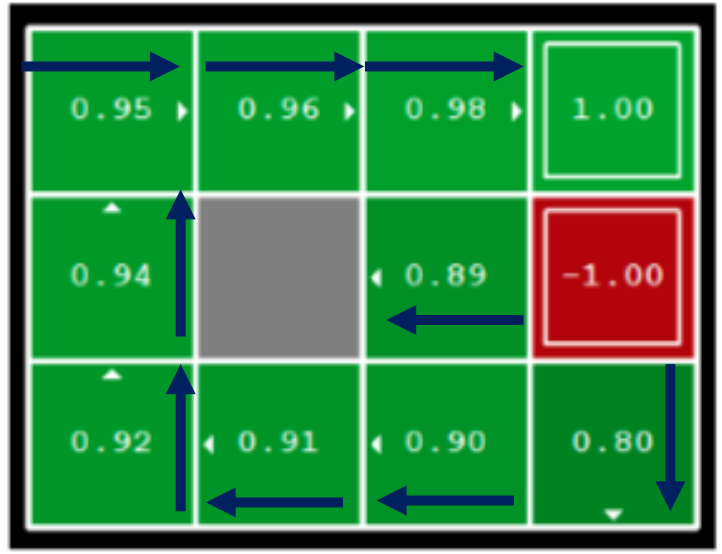
Value (utility) of a q-states Q(s,a) for all states and all actions at each state.



Noise = 0.2
Discount = 0.9
Living reward = 0

Extracting Policy from the Optimal Value Function

- **Our goal is to determine the policy for the MDP**



- **Step I: Estimate the optimal values V*(s)**
  - Through Value Iteration algorithm
- **Step II: Policy Extraction**
  - Obtain the policy implied by the values (using 1-step look ahead).
  - Use the policy to act in the environment.

$$\pi^*(s) = \arg\max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

More Efficiency:
In-place Dynamic Programming

- Dynamic programming methods (Bellman updates) used synchronous updates (all states in parallel)
- We can also update in-palace
  - backs up states individually, in any order and can significantly reduce computation. Guaranteed to converge if all states continue to be selected

- Synchronous value iteration stores two copies of value function

$$\text{for all } s \text{ in } \mathcal{S}: \quad v_{\text{new}}(s) \leftarrow \max_a \mathbb{E}\left[R_{t+1} + \gamma v_{\text{old}}(S_{t+1}) \mid S_t = s\right]$$

$$v_{old} \leftarrow v_{new}$$

- In-place value iteration only stores one copy of value function

$$\text{for all } s \text{ in } \mathcal{S}: \quad v(s) \leftarrow \max_a \mathbb{E}\left[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s\right]$$

More Efficiency: *Prioritized Sweeping*

- Use magnitude of Bellman error to guide state selection, e.g.

$$\left| \max_a \mathbb{E}\left[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s\right] - v(s) \right|$$

- Backup the state with the largest remaining Bellman error
- Update Bellman error of affected states after each backup
- Requires knowledge of reverse dynamics (predecessor states)
- Can be implemented efficiently by maintaining a priority queue

How to compute the “optimal” policy?
Policy iteration – iterate directly in the policy space

Policy Iteration: An alternative

Policy Iteration:

- Pick a random policy: a random assignment of actions to states.
- **Step 1: Policy evaluation:** calculate utilities for some fixed policy (not optimal utilities!) until convergence.
- **Step 2: Policy improvement:** update policy using one-step look-ahead with resulting converged (but not optimal) utilities as future values.
- **Repeat** steps until policy converges.

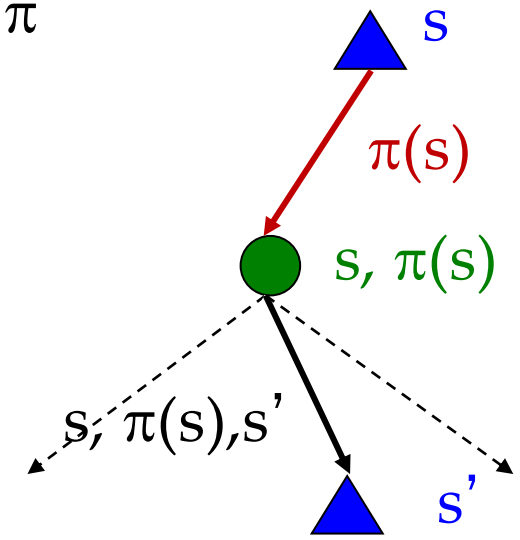
- Converges to the optimal policy. In practice, can converge faster*.

*conditions beyond the scope of this course.

Step I: Policy Evaluation

Evaluate the utilities for states under the actions provided by the given policy

- Policy Evaluation: Calculating the value function V for a fixed policy π



- Perform a number of simplified Bellman updates (given a policy)

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

- The Bellman equations are a linear system of equations without the max operator.
  - Can be solved in $O(n^3)$ time by standard linear algebra solvers.
  - It is not necessary to obtain the exact solution.
  - Can perform some number of simplified value iteration steps (simplified because the policy is fixed) to give a good approximation of the utility values of the states.

- Efficiency: $O(S^2)$ per iteration

Given evaluation, how to improve the policy?

- We say that $\pi \geq \pi'$ if $V^{\pi}(s) \geq V^{\pi'}(s) \forall s \in S$
- This gives a partial ordering of policies: if one policy is better at one state but worse at another state, the two policies are incomparable
- Since we know how to compute values for policies, we can search through the space of policies
- Local search seems like a good fit.

- Suppose that there is some action a^*, such that:

$$r(s, a^*) + \gamma \sum_{s' \in S} p(s, a^*, s') V^{\pi}(s') > V^{\pi}(s)$$

What if we decide to change the policy with this action.

Iteratively Improving the policy

- Suppose that there is some action a^*, such that:

$$r(s, a^*) + \gamma \sum_{s' \in S} p(s, a^*, s') V^{\pi}(s') > V^{\pi}(s)$$

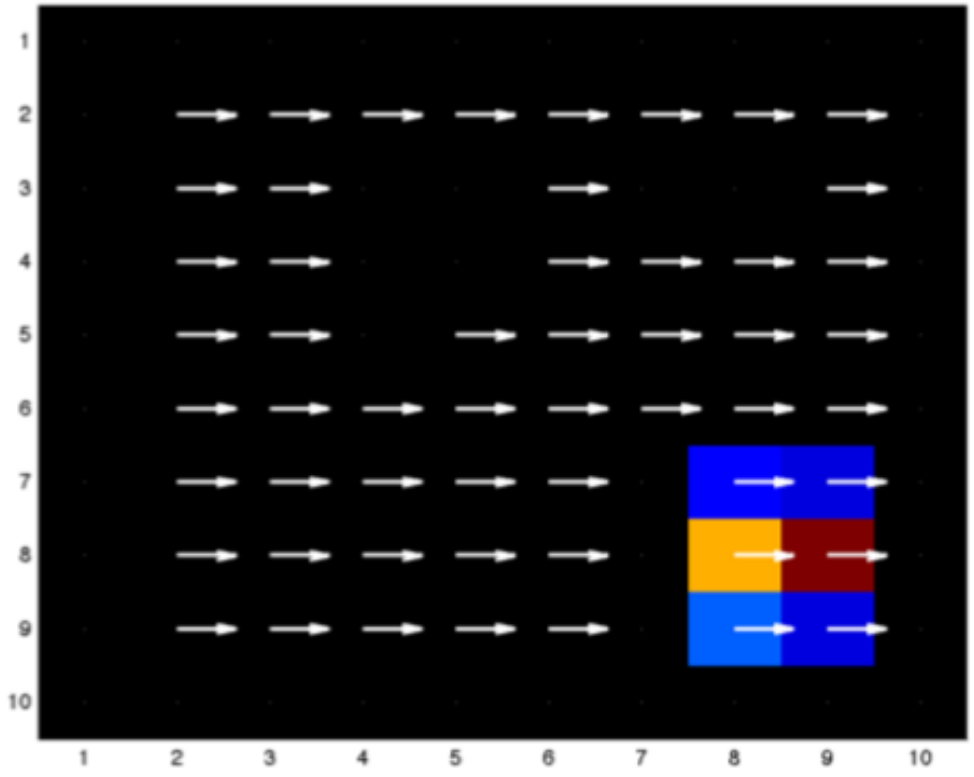
Consider:
If we set this action, the value of the state s will increase

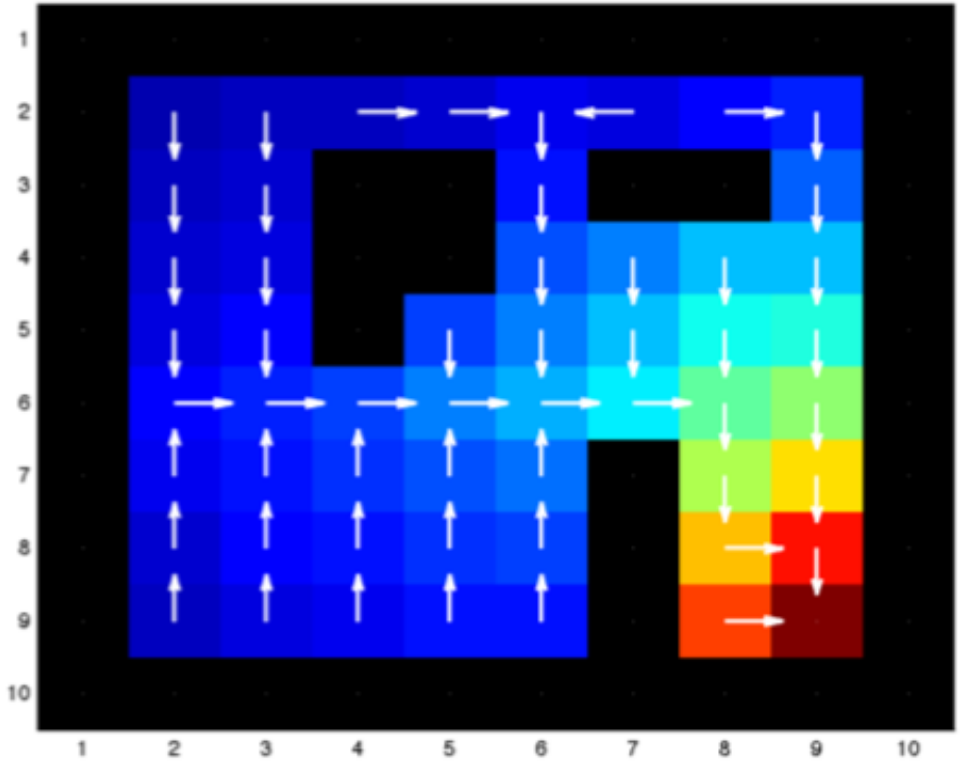
Why?

- This is because we replaced each element in the sum that defines $V^{\pi}(s)$ with a bigger value
- The values of states that can transition to s increase as well
- The values of all other states stay the same
- So the new policy using a^* is better than the initial policy π!

Iteratively improving the policy

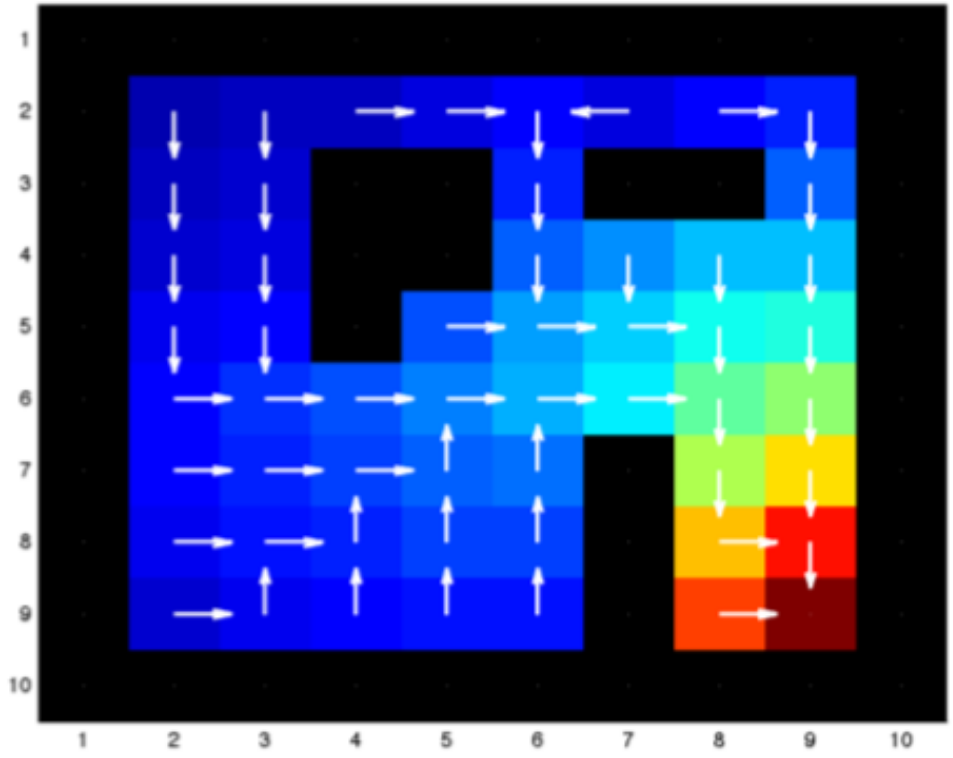
Interleaving policy evaluation and policy improvement. Start with a naïve policy.

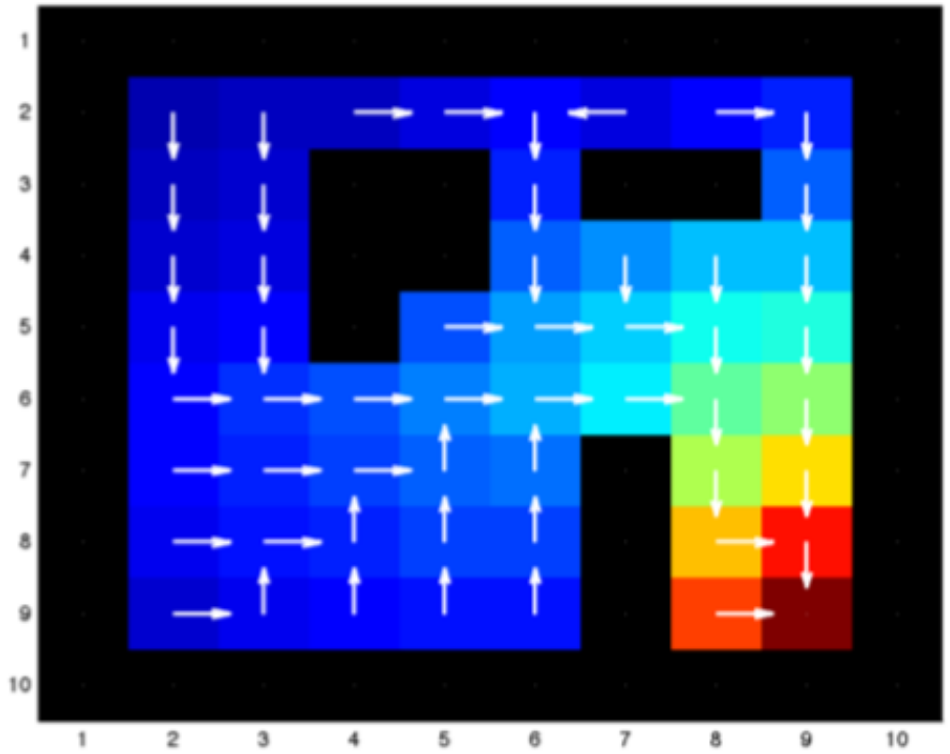




Iteratively improving the policy

Convergence in 4 iterations





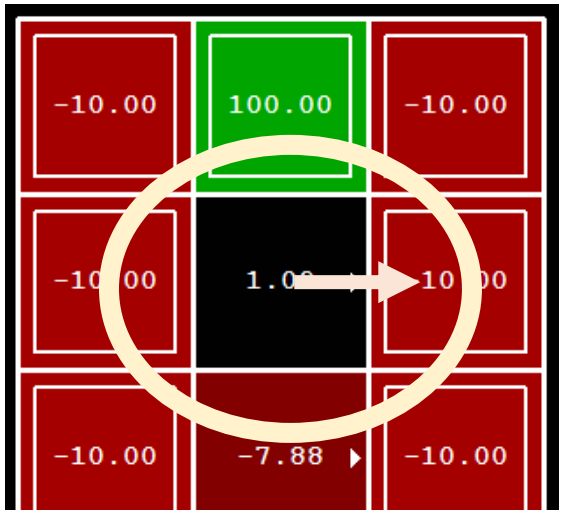
The procedure gives a way to do local search through the space of policies.

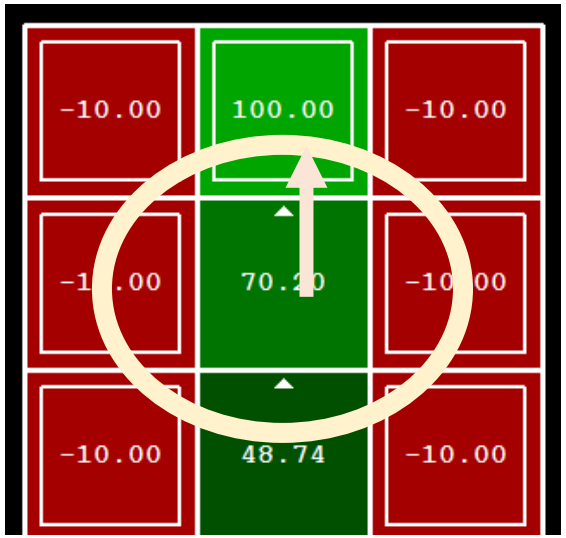
Step II: Policy Improvement

- Improvement
  - Given fixed utility values for states (obtained via policy evaluation)
  - Examine if there is a better policy (policy extraction) using one-step look-ahead
  - Greedily changing the policy.

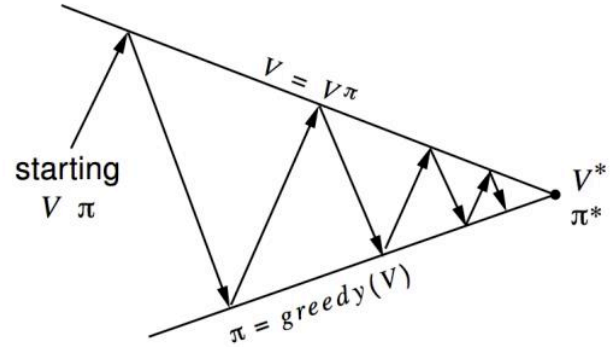
$$\pi_{i+1}(s) = \arg\max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^{\pi_i}(s') \right]$$

Toy example





Policy Iteration: Evaluation and Imprc



- **Pick an arbitrary policy**
- **Iterate (until the policy changes)**
  - **Policy Evaluation:** For fixed current policy π, find values with policy evaluation:
    - $$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') \left[R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s') \right]$$
  - **Policy Improvement:** For fixed values, get a better policy using policy extraction
    - One-step look-ahead:
      $$\pi_{i+1}(s) = \arg\max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^{\pi_i}(s') \right]$$

Policy Iteration: Pseudocode

- Consider improving the policy
  - Given the fixed utility values for states (obtained via policy evaluation)
  - Examine if there is a better policy using one step look ahead.

Algorithm 2 An algorithm for computing an MDP policy by repeated evaluation and improvement.

POLICY-ITERATION$(\mathcal{S}, \mathcal{A}, R, T, \gamma, \epsilon)$

```
1  for each state s ∈ S
2      i = rand(0, |A|)
3      π[s] = a_i
4  t = 0
5  V_0 = POLICY-EVALUATION(S, π, R, T, γ, ε)
6  repeat
7      change = 0
8      t = t + 1
9      for each state s ∈ S
10         π_t[s] = argmax_a (Σ_s' T(s, a, s') [R(s, a, s') + γ · V_{t−1}(s')])
11     V_t = POLICY-EVALUATION(S, π, R, T, γ, ε)
12     for each state s ∈ S
13         change = change + V_t[s] − V_{t−1}[s]
14 until change < ε
```

Key Idea:

- Evaluate and improve the policy.
- Interleave evaluation and improvement

Takeaways

- Sequential decision making under action uncertainty
  - MDP formulation
  - Given: States, Actions, Rewards, Discounting, Transition Function
  - Allows determination of the optimal policy.
- Key Assumptions
  - Transition function is known – We know (analytically or via a probability distribution) the possible consequences of actions.
  - We know what reward we will get at any state/state transition in the MDP. Essentially, we know where the good states are.
  - **Under these assumptions we can compute the optimal behaviour** for the autonomous system.

But, what happens when action consequences and reward function is not known ahead of time (or is too complex) to even write? We will understand this in the next topic.